Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

Ho Chi Minh City, 02 November 2026

Table of Contents

I. Record of Changes 3
II. Software Design Document 3
1. System Design 3
1.1 System Architecture 3
1.1.1 Authority and Data Boundaries 5
1.1.2 Main Interface Contracts 5
1.1.3 Deployment View (Logical) 5
1.1.4 Technology, DevOps and External Services 6
1.2 Package Diagram 6
2. Database Design 7
2.1 Data Storage Strategy 7
2.2 Relational Data Model (PostgreSQL) 7
2.2.1 Table Descriptions – Accounts and Economy 9
2.2.2 Table Descriptions – Match, Reward and AI 9
2.2.3 Additional Database Entity Coverage and Scope 9
2.2.4 Persistence Integrity Rules 10
2.3 Telemetry Document Model (MongoDB) 10
3. Detailed Design 10
3.1 Account Authentication and Profile 10
3.2 Multiplayer Lobby and Session Lifecycle 14
3.3 Player Interaction, Life State and Revive 16
3.4 Expedition Mission and Objective Control 19
3.5 Stalker Monster AI 22
3.6 Adaptive Experience Director and Telemetry 24
3.7 Match Result and Reward Processing 26
3.8 Virtual Shop, Wallet and Inventory 26
3.9 UI, HUD and Gameplay Feedback 28
3.10 Main Business Workflows 29
3.10.1 WF-01 – Authentication, Legal Agreement and Lobby 30
3.10.2 WF-02 – Expedition Gameplay and Match Outcome 33
3.10.3 WF-03 – Stalker AI Gameplay and Player Survival 36
3.10.4 WF-04 – Match Result, Reward and Virtual Economy 39
4. Art Direction and Moodboard (Game Projects) 42
4.1 Visual Summary and Pillars 42
4.2 Overall Mood 42
4.3 Color Palette and Readability Rules 43
4.4 Lighting and Atmosphere 43
4.5 Environments and Locations 43
4.6 Characters and Factions 44
4.7 Items, Props and Objects 45
4.8 Animation 45
4.9 UI and HUD Style 46
4.10 VFX 46
4.11 Key Art and Store Assets 47
4.12 Reference Sources and Asset Licences 47
5. Sound Design (Game Projects) 48
5.1 Audio Vision and Pillars 48
5.2 Platform and Audio Constraints 48
5.3 Audio Direction and References 48
5.4 Gameplay Sound Effects 49
5.5 Character, Environment and Ambience 49
5.6 Music 50
5.7 UI and Feedback Sounds 50
5.8 Voice-over and Localization 50
5.9 Mixing 51
5.10 Technical Specifications 51
5.11 Audio Settings, Asset List and Schedule 51

I. Record of Changes

Date	A/M/D	In charge	Change Description
28/10/2026	A	ECHO PROTOCOL team	Initial SDD using the school Report 4 outline, Report 1–3 and the inspected GitHub source. Player participation scope fixed at 1–4.
30/10/2026	M	Lâm Thanh Phúc	Selected LTP source; added versioned Legal Agreements lifecycle and consent entity, corrected Adaptive AI/fixed fallback boundary; registry unchanged.
31/10/2026	M	Nguyễn Nhật Lệ Thuyên	Corrected interaction class descriptions, aligned LTP technology and deployment references, and added the agreement acceptance ERD extension.
31/10/2026	A	ECHO PROTOCOL team	Added four main business workflows (WF-01–WF-04); WF-03 documents Stalker gameplay and player survival.
02 Nov 2026	M	ECHO PROTOCOL team	Aligned Report 4 with the updated faculty game template and the existing GDD; added Art Direction/Moodboard and Sound Design with project visuals and source-linked audio/asset inventories; preserved existing technical diagrams.
03 Nov 2026	M	ECHO PROTOCOL team	Re-verified Sections 4–5 against current main @ 02ada8537795; corrected UI palette, typography, scene/audio event mapping and verification status. Sections 1–3 retain their LTP design baseline.

A = Added; M = Modified; D = Deleted.

II. Software Design Document

1. System Design

1.1 System Architecture

ECHO PROTOCOL separates real-time session state from persistent backend state. Unity owns player presentation and local input; Photon Fusion Host Mode provides network session connectivity, while the session Host validates critical gameplay changes. ASP.NET Core handles authenticated APIs, game results, rewards, wallet operations, inventory, and AI records. PostgreSQL is the transactional store, and MongoDB stores gameplay telemetry.

The mandatory Review 2 architecture consists of the Unity Windows client, Photon Fusion Host-authoritative gameplay, the ASP.NET Core backend, PostgreSQL and MongoDB persistence, and the limited administration interface.

The implementation additionally contains a Player Web Portal and a payOS payment integration. The Player Web Portal communicates with the backend through authenticated HTTP APIs. Payment checkout requests and verified webhook callbacks are processed through backend payment services rather than directly through the Unity gameplay client.

These additional components are included in Figure 1 for architecture completeness but are outside the mandatory Review 2 acceptance scope. Their presence in source code does not imply completed production payment readiness or successful end-to-end acceptance testing.

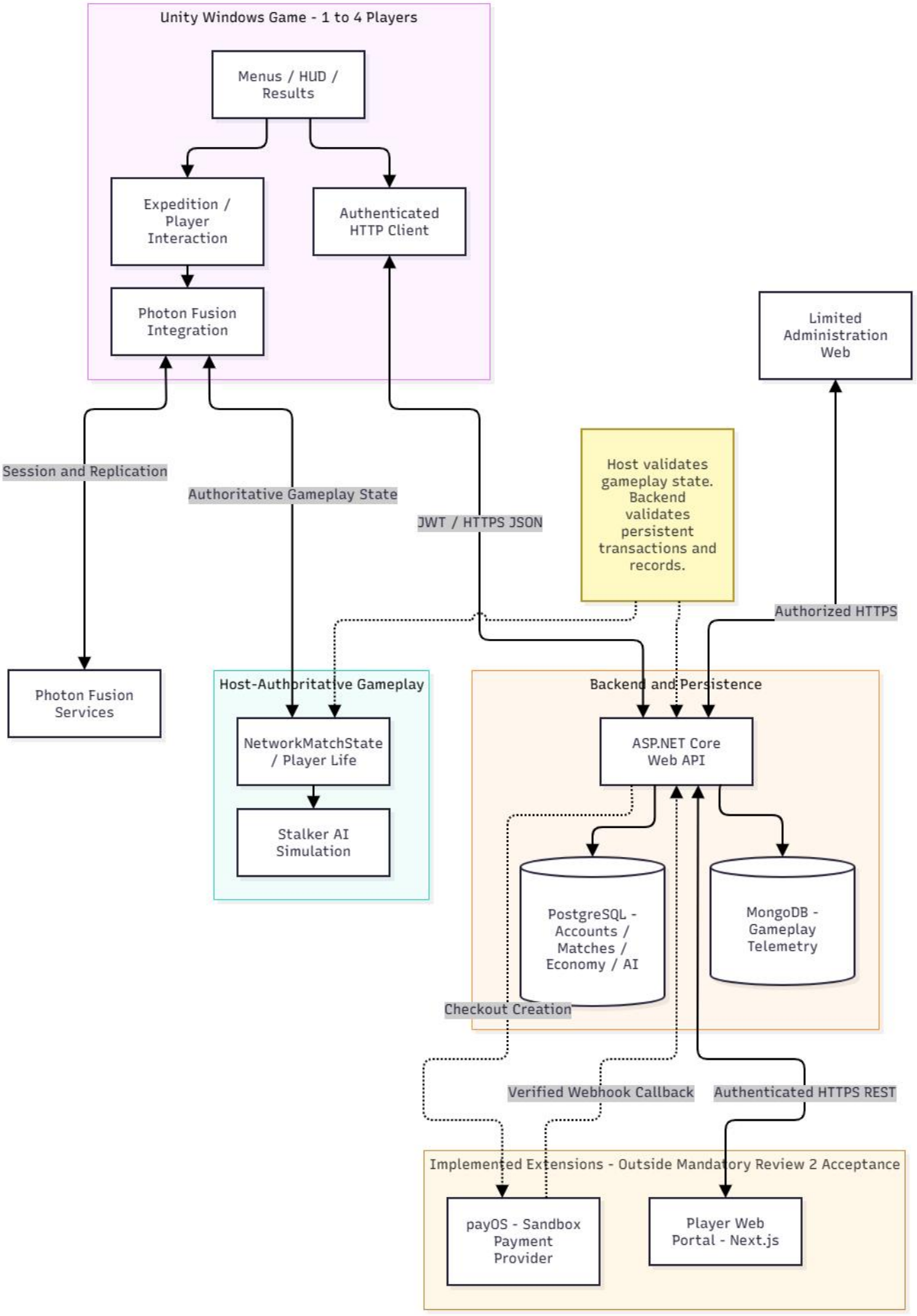


Figure 1. Overall logical architecture and external interfaces

Component	Key responsibility	Integration
Unity client	Input, player actions, expedition controllers, AI presentation, HUD, menu, result and shop	Photon networking; HTTPS API
Photon Fusion / Host	Session membership, network runner, replicated state; host-only authoritative gameplay mutation	Fusion protocol / callbacks / RPC
ASP.NET Core API	Authentication, authority binding, result validation, reward, shop, roles, scenario records	HTTPS JSON / JWT
PostgreSQL	Relational identity, progression, economy, match and scenario records	EF Core / Npgsql
MongoDB	Backend-ingested gameplay event telemetry.	MongoDB driver / repository
Admin web	View limited protected user/economy data and permitted auditable changes	Authenticated Backend API

1.1.1 Authority and Data Boundaries

Player requests do not directly own final mission state. Locally collected input or interaction commands are validated at State Authority before any corresponding replicated state is changed. Client HUD and effects read the accepted Networked state. The Backend must independently validate Host identity and registered match before committing results, rewards, and wallet transactions; a client display is not evidence of server acceptance.

1.1.2 Main Interface Contracts

Interface	Transport / Authority	Design constraints
Unity <-> Photon Fusion	NetworkRunner, NetworkBehaviour, RPC, Networked state	Host validation; no direct arbitrary client commit of objective/result
Unity / authorized Host <-> Backend	HTTPS REST + JWT	Validate identity/roster/permissions; return explicit rejection
Backend <-> PostgreSQL	EF Core transactions	Consistency, unique keys, idempotency for reward/purchase
Backend <-> MongoDB	Telemetry repository	Match identity, event sequence, timestamps and schema version
Admin Web <-> Backend	Authorized HTTPS API	RBAC enforced server-side, not only hidden UI controls
Unity/Web <-> Agreement API	HTTPS JSON/JWT; GET documents/status, POST accept	Current bundle/hash match; all required flags; 403 AGREEMENT_REQUIRED for protected operations; no cash/payment obligation added.

1.1.3 Deployment View (Logical)

ECHO PROTOCOL runs as a Windows Unity client connected to Photon Fusion Host Mode and an ASP.NET Core backend deployed on AWS EC2. GitHub Actions supports build and release workflows, while Docker Compose manages backend, web and database services. PostgreSQL and MongoDB connection settings are supplied by environment configuration. The inspected LTP documentation baseline is 08de8d2347d118077a076ba032568236efac4898 (31 October 2026); the final Review 2 tag must reference the later approved document commit.

1.1.4 Technology, DevOps and External Services

Technology, integration services and workflows for the reviewed LTP baseline are listed below.

CATEGORY	VERSION / SERVICE	PURPOSE / DATA / VERIFICATION
Game engine	Unity 6000.5.8f1; C#; Photon Fusion Host Mode	Windows game, network session membership, replicated gameplay; Photon service receives session/connection data.
Backend / web	.NET 8 / ASP.NET Core; Next.js 16.4.3	Authenticated REST APIs; limited Admin UI. JSON/JWT, authorization and explicit errors.
Databases	PostgreSQL (EF Core/Npgsql 8.0.11); MongoDB.Driver 3.11.0	Relational user, match, reward and economy records; MongoDB event telemetry. Server versions depend on deployment.
Deployment / CI	GitHub Actions; Docker Compose; AWS EC2	GitHub Actions pipelines build and check backend/Unity code; Docker Compose supports AWS EC2 deployment and operational scripts.
External services	Photon Fusion; AWS	Photon Fusion handles network sessions; AWS hosts backend/data services.
AI tools used	OpenAI ChatGPT (GPT-6)	Assisted with requirements, documentation, diagram/source review and test planning; project team verifies technical decisions.

For the mandatory Windows client, the Unity engine provides rendering, animation, physics, input and audio. Scene/bootstrap and local settings coordinate entry; gameplay and mission controllers handle the player, equipment, three-zone Expedition, objectives and Stalker AI; presentation controllers expose HUD, feedback and SFX; Photon Fusion carries the replicated Host-authoritative session. This is a logical responsibility view, not a claim of separate assemblies. See the integrated GDD in Report 3 Section 3.4 and the art/audio specifications in this report Sections 4–5.

1.2 Package Diagram

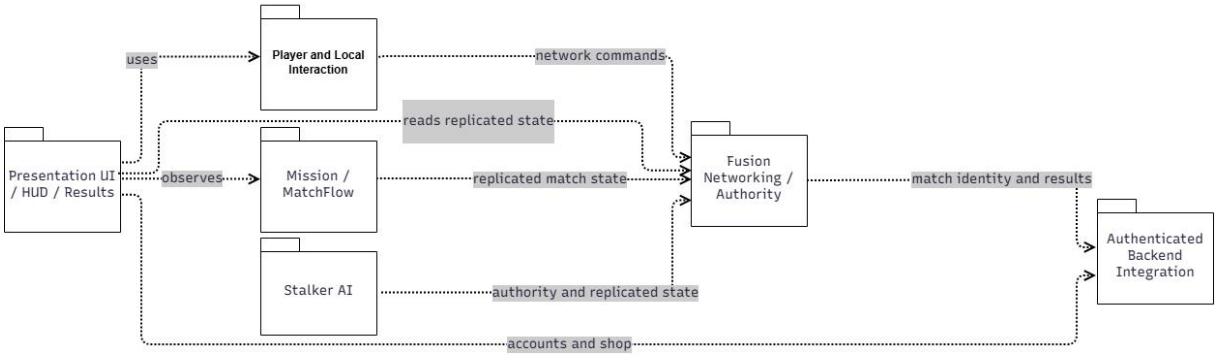


Figure 2. Unity gameplay and networking packages

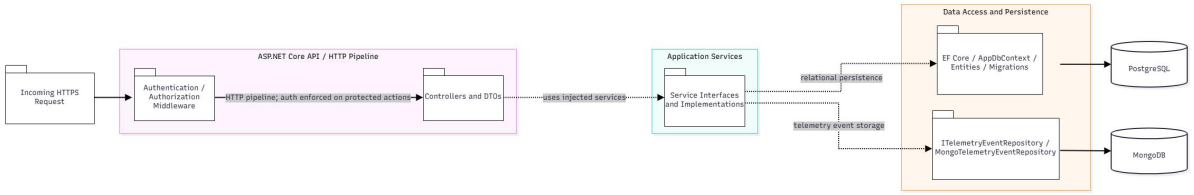


Figure 3. Backend and persistence packages

No.	Package	Description / source area
01	Presentation	MainMenu, Lobby UI, HUD, Result and shop presentation components.
02	Player & Interaction	Handles local player input, equipment and interaction prompts.
03	Mission & Match Flow	MatchFlowController, energy cores, objectives, power/security, evacuation.
04	Stalker AI	StalkerController, StalkerVisionSensor, StalkerFusionRuntime, scenario resolution.
05	Networking & Authority	Manages Fusion sessions, RPCs, player life state, replicated match state and Host authority.
06	Backend API / Services	Controllers, DTOs, authentication, match, reward, purchase, inventory and admin services.
07	Persistence & Telemetry	AppDbContext entities/migrations; MongoTelemetryEventRepository.

2. Database Design

2.1 Data Storage Strategy

PostgreSQL is the authoritative source for persistent user and transactional records. Tables are mapped with Entity Framework Core via AppDbContext. MongoDB stores event-oriented telemetry records used by analysis and adaptive inputs. Photon replicated state (e.g., current mission phase, player life and enemy state) is transient session state and should not be modeled as a database table unless it is explicitly persisted by a backend contract.

Game save boundary: PlayerPrefs records local non-authoritative options (input bindings, graphics, audio and UI language). PostgreSQL persists identities, match/reward records, agreements, wallet and inventory; MongoDB stores telemetry. Mission and Stalker active states are synchronized by Host/Photon Fusion and are not authoritative local save files. Data changes are versioned through database migrations and project-controlled configuration assets; a separate generic JSON/cloud-save migration mechanism is not asserted.

2.2 Relational Data Model (PostgreSQL)

The PostgreSQL logical model separates account/economy entities (Figure 4) from match, reward and adaptive-decision entities (Figure 5). A versioned acceptance record, UserAgreementAcceptances, references Users by UserId and uses the composite key (UserId, BundleVersion)

Figures 4 and 5 present selected logical views of the PostgreSQL data model for the mandatory Review 2 scope. Figure 4 focuses on account authentication, legal acceptance, virtual economy, and inventory ownership, while Figure 5 focuses on authoritative matches, rewards, player profiles, and bounded Adaptive AI records.

These figures do not represent every table created by Entity Framework Core migrations. Additional payment integration, operational recovery, and experimental Adaptive AI ledger tables are retained in the implementation schema but are documented separately according to their responsibilities and Review 2

acceptance status. Their presence in the database does not, by itself, establish that the corresponding functionality has passed acceptance testing.

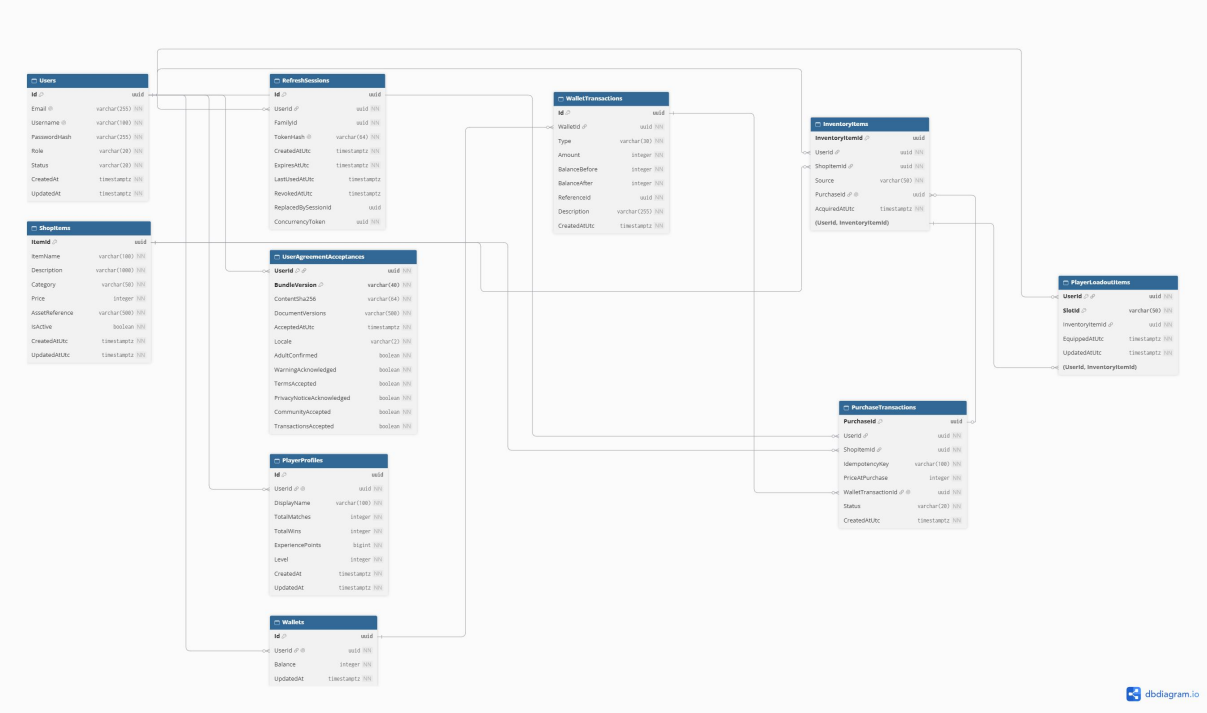


Figure 4. Core account and virtual economy ERD (logical cardinalities)

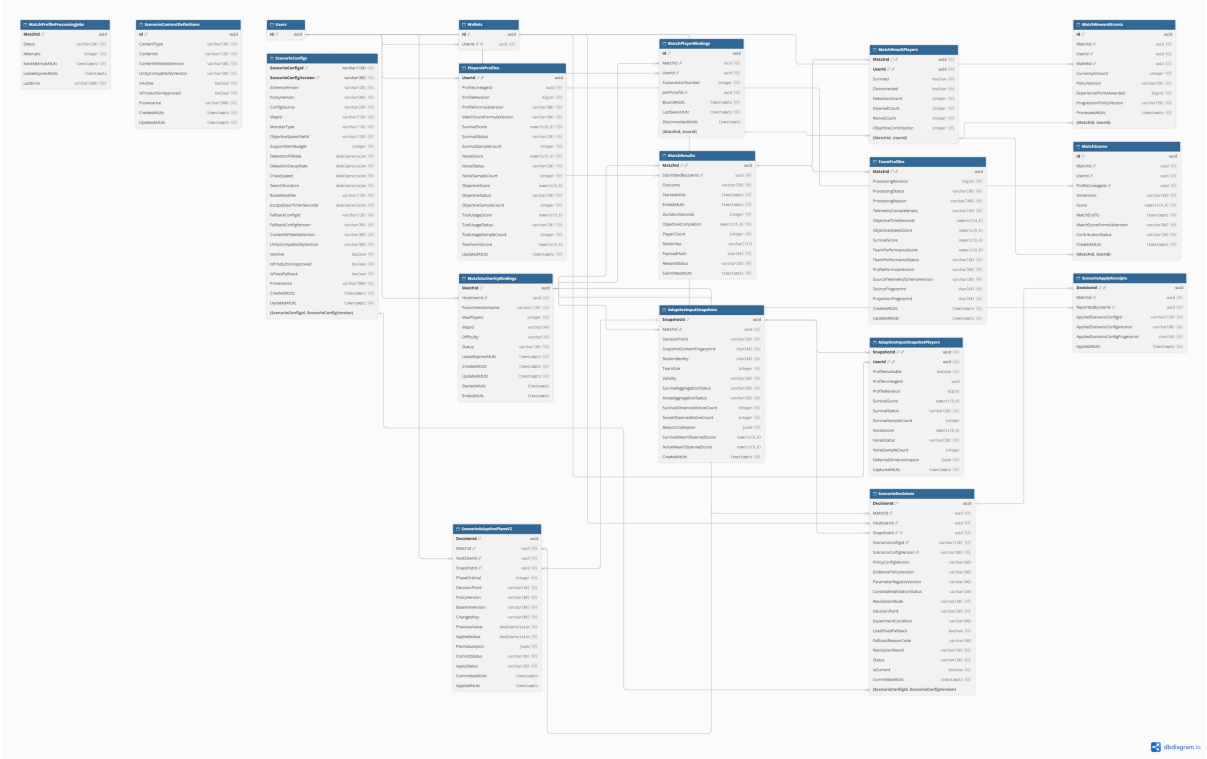


Figure 5. Core match, reward and adaptive-record relationships

2.2.1 Table Descriptions – Accounts and Economy

Table	Purpose / key fields

Table	Purpose / key fields
Users	PK Id; account identity, hashed password, role and status.
RefreshSessions	Authentication refresh sessions; user reference, token hash and expiry/concurrency constraints.
PlayerProfiles	Account progression and profile data; one-to-one user association.
Wallets	PK Id, FK UserId; persistent ECHO Credits balance.
WalletTransactions	PK Id, FK WalletId; type, amount, before/after balance, reference identity.
ShopItems	Eligible server-side catalogue, price, status and category.
PurchaseTransactions	PK PurchaseId, FK UserId, ShopItemId, WalletTransactionId; IdempotencyKey.
InventoryItems	PK InventoryItemId, FK UserId, ShopItemId, optional PurchaseId; ownership source.
PlayerLoadoutItems	Account equipment selections referencing owned compatible inventory. PK (UserId, SlotId); composite FK to owned inventory.
UserAgreementAcceptances	UserId FK → Users; composite PK (UserId, BundleVersion). Version/hash, document versions, UTC acceptance, locale and six acknowledgement flags; added by LTP migration 20261030090842.

2.2.2 Table Descriptions – Match, Reward and AI

Table	Purpose / key fields
MatchAuthorityBindings	PK MatchId; Host UserId, room binding, status, lease and selected match configuration.
MatchPlayerBindings	Participant identity/roster binding associated with authoritative match.
MatchResults	PK MatchId; authoritative match outcome, duration, submitted-by user and roster identity.
MatchResultPlayers	PK (MatchId, UserId); per-player match result and objective/survival statistics.
MatchRewardGrants	PK Id; MatchId, UserId, WalletId, CurrencyAmount, XP and policy versions.
MatchScores	Validated match scoring records associated with players/results.
PlayerAIProfiles	PK/FK UserId; aggregated player behaviour/profile data for AED inputs.
TeamProfiles	PK/FK MatchId; aggregated team-level behaviour/result data.
ScenarioConfigs / ScenarioContentDefinitions	Versioned allowed scenario configurations and content.
AdaptiveInputSnapshots / AdaptiveInputSnapshotPlayers	Pre-match observed input snapshot and constituent players. PK (SnapshotId, UserId); player snapshot data.
ScenarioDecisions / ScenarioApplyReceipts	Decision identity, version/fingerprint, fallback reason and application receipt. PK/FK DecisionId; records scenario application.

2.2.3 Additional Database Entity Coverage and Scope

Database table	Purpose	Review 2 scope	ERD reference
MatchScores	Stores per-match score records used in player profiling	Supporting Adaptive AI evaluation data	Figure 5
ScenarioConfigs	Stores versioned, bounded scenario parameters and Fixed Fallback configurations	Mandatory Adaptive AI configuration support	Figure 5
ScenarioContentDefinitions	Defines approved map, monster, objective spawn-set and route-modifier identities	Supporting scenario compatibility validation	Figure 5
PaymentOrders, PaymentCheckouts, PaymentProviderEvents, PaymentFulfillments	Stores external payment checkout, callback and fulfillment records	Implemented extension; outside mandatory Review 2 acceptance	Migration schema only
ScenarioAdaptivePlansV2	Stores additional adaptive scenario planning decisions	Additional AED V2 planning; not evidence of complete acceptance	Figure 5
AEDDirectorDecisionsV1,	Stores experimental director	Supporting/research implementation;	Migration schema only

Database table	Purpose	Review 2 scope	ERD reference
AEDPhase8VerifiedSourcesV1	decisions and verification records	outside mandatory acceptance unless separately verified	
AEDV02Decisions, AEDV02SceneContents, AEDV02SkillObservations, AEDV02VerifiedSources, AEDV02Cancellations	Stores additional AED V2 runtime/verification records	Experimental or additional implementation; outside mandatory acceptance unless separately verified	Migration schema only

The entity coverage table distinguishes mandatory Review 2 data responsibilities from additional implementation and research records. MatchScores and ScenarioContentDefinitions are included in the updated Figure 5 to complete the selected logical relationships. Payment and additional AED ledger tables remain documented through the migration schema without expanding the mandatory acceptance scope.

The PostgreSQL migration script is the reference for actual table names, keys, constraints, and foreign-key relationships. The ERD visualization SQL is a selected documentation view and must not be executed as a production database migration.

2.2.4 Persistence Integrity Rules

- Wallet balances must not be trusted from Unity input. PurchaseService checks eligible item, ownership and available balance, and creates ledger + purchase + inventory grant within a database transaction.
- A repeated purchase IdempotencyKey must not charge Credits again or duplicate ownership. Conflicting replay requests must be rejected.
- MatchResultService checks Host authorization, active match state, roster and accepted result identity before committing. RewardService grants the corresponding reward once, with replay handling.
- Physical constraints and indexes are defined in EF Core model configuration and migrations.

2.3 Telemetry Document Model (MongoDB)

Field	Type / role
Id	Guid; event document identity
MatchId	Guid; match correlation
UserId	Nullable Guid; user correlation if applicable
EventType	String; event classification
Ts	UTC timestamp of source event
EventSequence	Long; authoritative event ordering
ValueJson	BSON document; event payload
ReasonCode	Optional string; explanation for outcomes
SchemaVersion	String; compatibility/version
SemanticFingerprint	String; replay/content identity
IngestedAt	UTC ingestion timestamp

3. Detailed Design

3.1 Account Authentication and Profile

After authentication, Unity/Web requests agreement status and current documents through GET /api/agreements/status and GET /api/agreements/documents. The player accepts the matching bundle version/hash and required declarations via POST /api/agreements/accept. UserAgreementAcceptances stores the accepted version, locale, time and flags. AgreementGateMiddleware blocks protected match, scenario, write-inventory and purchase operations with 403 AGREEMENT_REQUIRED until consent is valid.

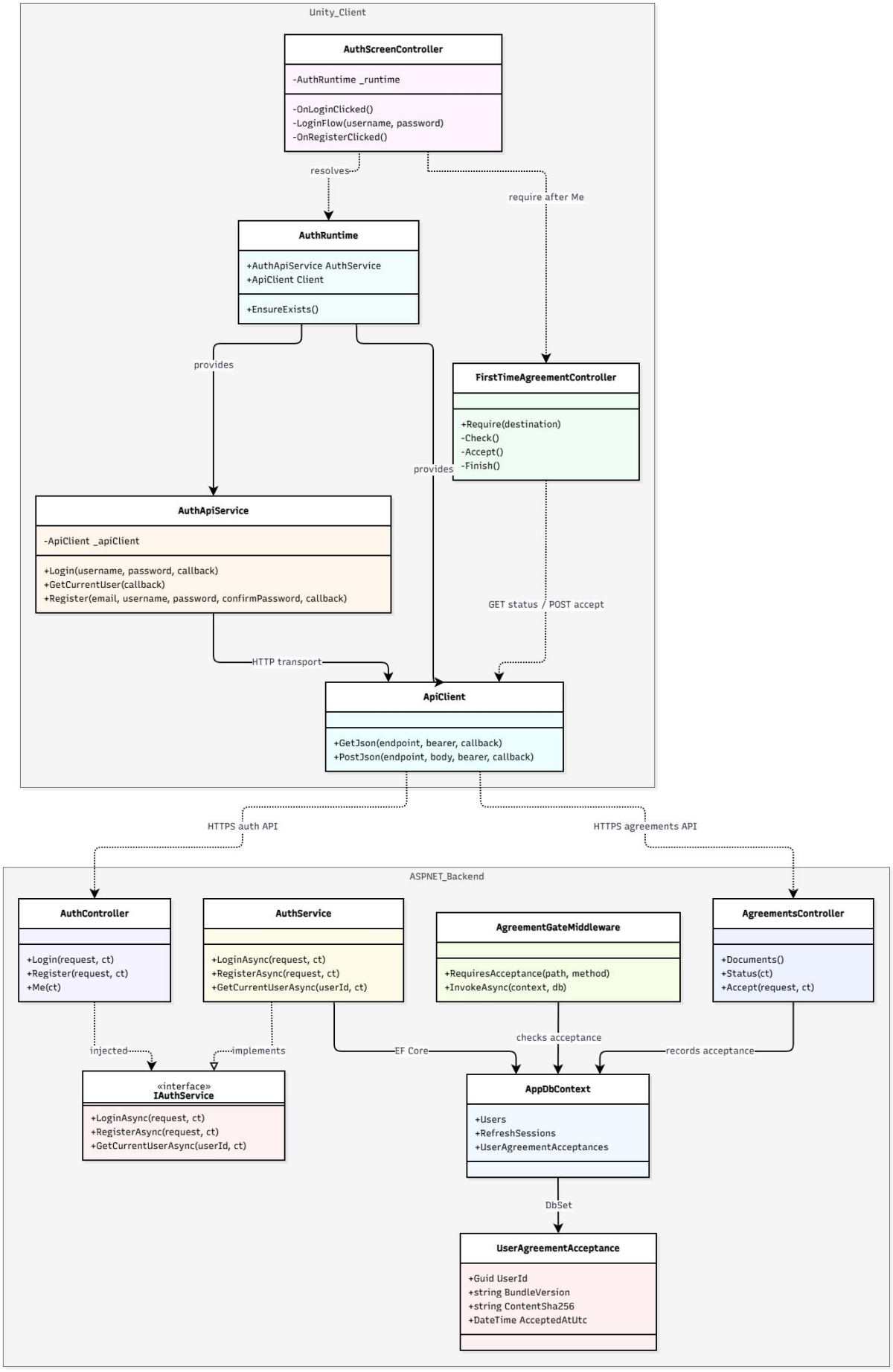

Unity_Client
AuthScreenController
-AuthRuntime _runtime
-OnLoginClicked()
-LoginFlow(username, password)
-OnRegisterClicked()
resolves
require after Me
AuthRuntime
+AuthApiService AuthService
+ApiClient Client
+EnsureExists()
provides
provides
FirstTimeAgreementController
+Require(destination)
-Check()
-Accept()
-Finish()
AuthApiService
-ApiClient _apiClient
+Login(username, password, callback)
+GetCurrentUser(callback)
+Register(email, username, password, confirmPassword, callback)
GET status / POST accept
HTTP transport
ApiClient
+GetJson(endpoint, bearer, callback)
+PostJson(endpoint, body, bearer, callback)
HTTPS auth API
HTTPS agreements API
ASPNET_Backend
AuthController
+Login(request, ct)
+Register(request, ct)
+Me(ct)
AuthService
+LoginAsync(request, ct)
+RegisterAsync(request, ct)
+GetCurrentUserAsync(userId, ct)
AgreementGateMiddleware
+RequiresAcceptance(path, method)
+InvokeAsync(context, db)
AgreementsController
+Documents()
+Status(ct)
+Accept(request, ct)
injected
implements
EF Core
checks acceptance
records acceptance
«interface»
IAuthService
+LoginAsync(request, ct)
+RegisterAsync(request, ct)
+GetCurrentUserAsync(userId, ct)
AppDbContext
+Users
+RefreshSessions
+UserAgreementAcceptances
DbSet
UserAgreementAcceptance
+Guid UserId
+string BundleVersion
+string ContentSha256
+DateTime AcceptedAtUtc

Figure 6. Selected authentication class/component diagram

Class / Component	Design responsibility / selected operations
AuthScreenController / AuthRuntime	Handles authentication UI and initializes Unity authentication services.
AuthApiService / ApiClient	Manages authentication requests, tokens and HTTP communication.
AuthController	Exposes protected authentication endpoints.
IAuthService / AuthService	Validates credentials and manages token issuance and refresh.
AppDbContext / User	Persists user identity, account status and authentication records.

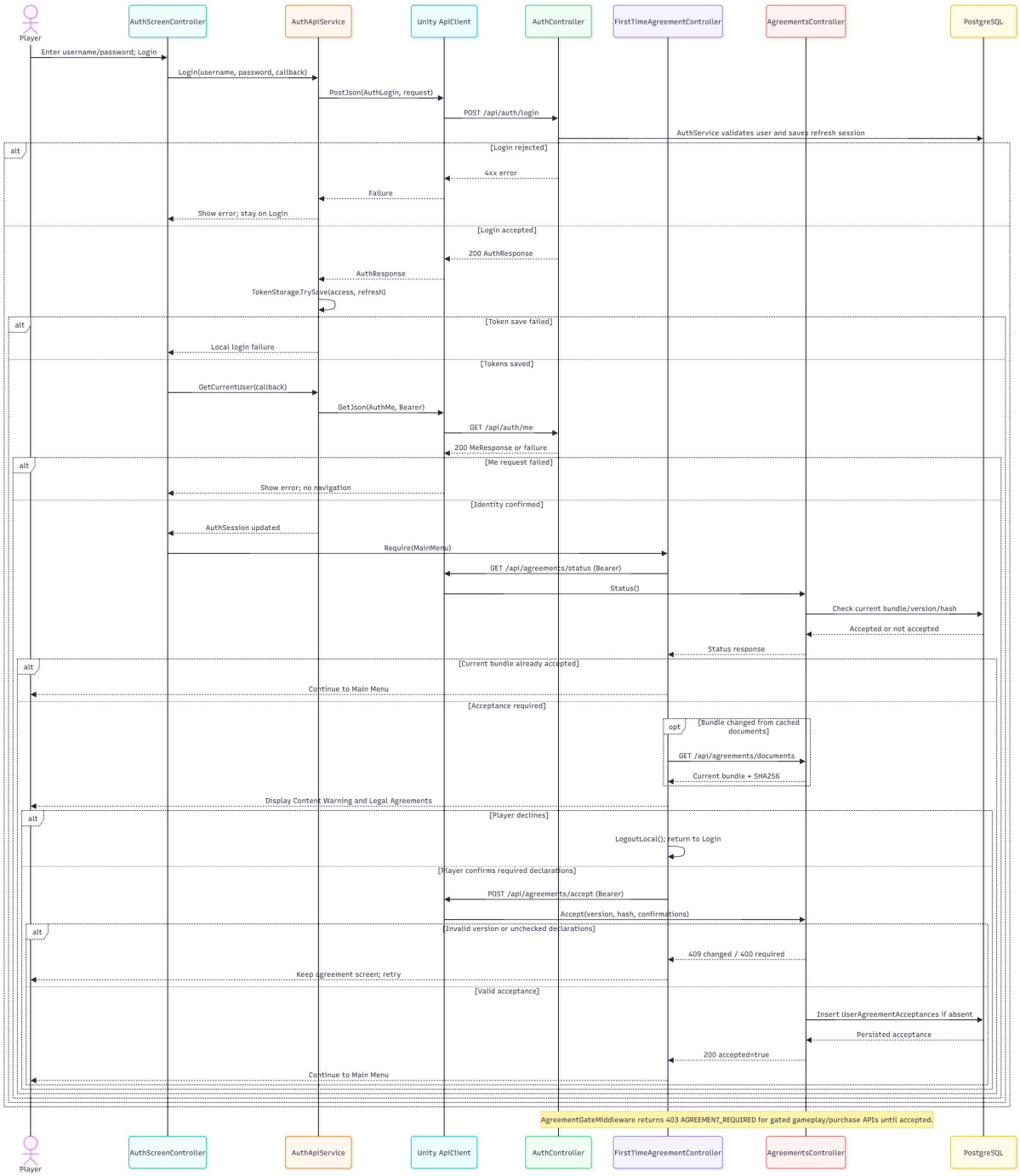


Figure 7. Login and authenticated session sequence

3.2 Multiplayer Lobby and Session Lifecycle

Field	Specification
Purpose	Create/join a private room, synchronize player readiness and start a host-authoritative Expedition.
Inputs / triggers	Room name, maximum room capacity, ready flags, selected difficulty/mode and Start input.
Authority / owner	NetworkBootstrap owns the runner; LobbyManager requests and validates Host-only start; MatchAuthorityRuntime binds backend authority.
Failure / negative path	Reject missing session, full/invalid room, non-host start, missing ready state or backend binding failure; never assume success on a failed transition.
SRS trace	FR-MUL-01–05; UC-03–05

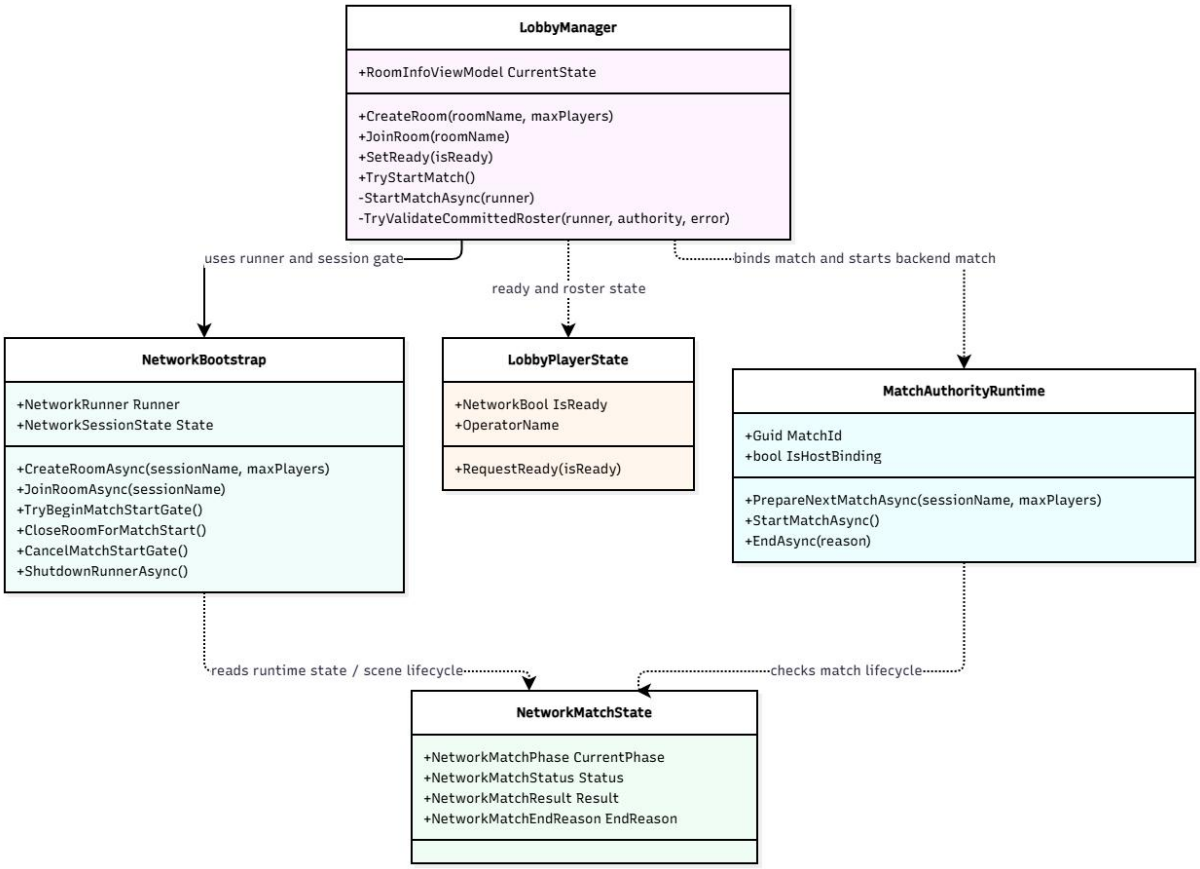


Figure 8. Lobby, runner and authoritative match state classes

Class / Component	Design responsibility / selected operations
NetworkBootstrap	NetworkRunner lifecycle; CreateRoomAsync, JoinRoomAsync, disconnect/reconnect and scene transitions.
LobbyManager / LobbyPlayerState	Lobby UI projection, ready and host-controlled match-start gate.
MatchAuthorityRuntime	Authoritative match binding and backend start flow.
NetworkMatchState	Replicated match state, phase and terminal outcome.

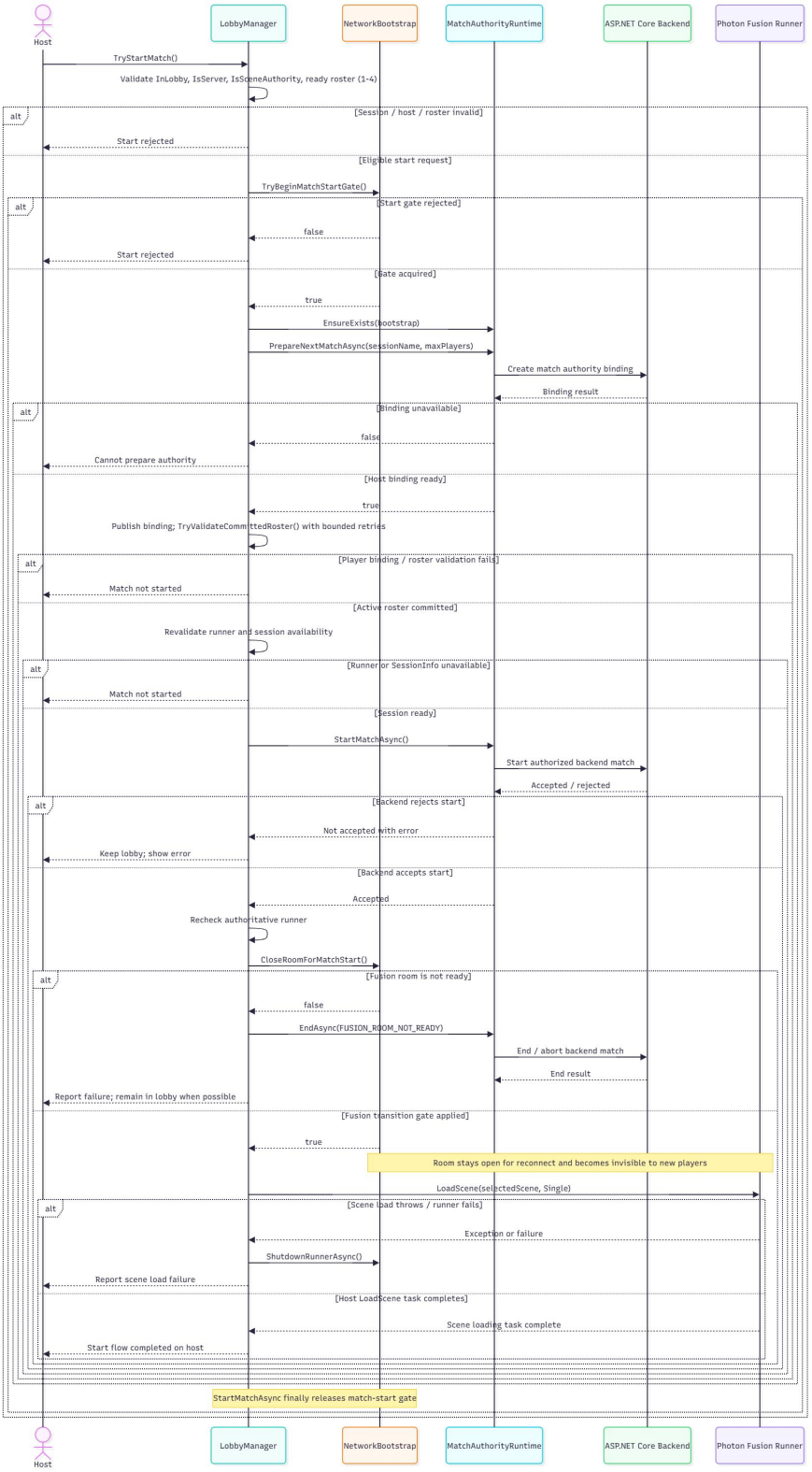


Figure 9. Host starts a private Expedition sequence (simplified)

Expedition supports 1–4 active players, including solo Host. The room capacity and number of active participants are separate; a Host-only start gate verifies membership/readiness and backend match authority before transitioning to gameplay.

3.3 Player Interaction, Life State and Revive

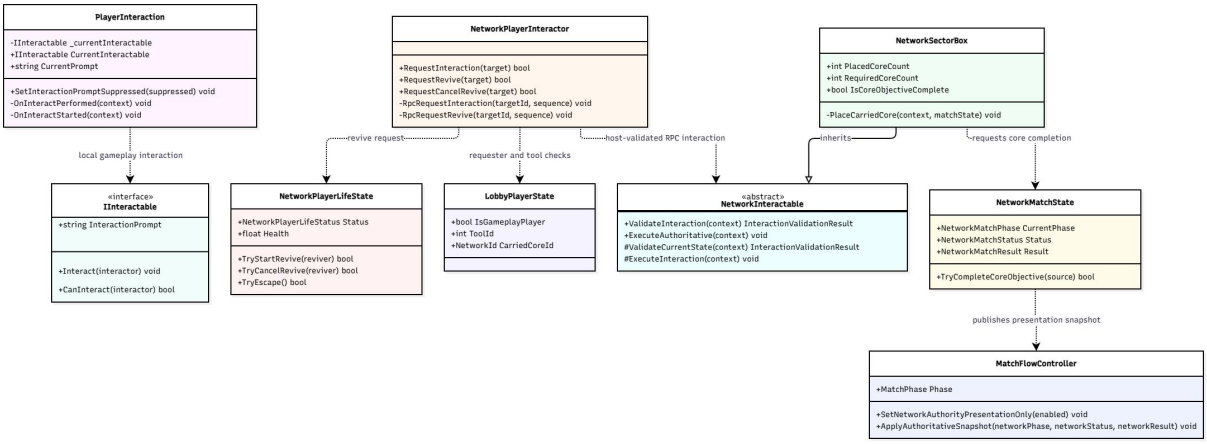


Figure 10. Selected player, objective and authoritative gameplay classes

Class / Component	Design responsibility / selected operations
PlayerInteraction / IInteractable	Detect eligible raycast target, issue interaction and publish prompt.
NetworkPlayerLifeState / Rules	Alive, Downed, Eliminated, Escaped and Caught; validate revive and damage.
NetworkMatchState	Game-wide authoritative state, ending and integration.
NetworkPlayerInteractor / NetworkInteractable	Sends and validates authoritative network interaction requests through Fusion RPCs.

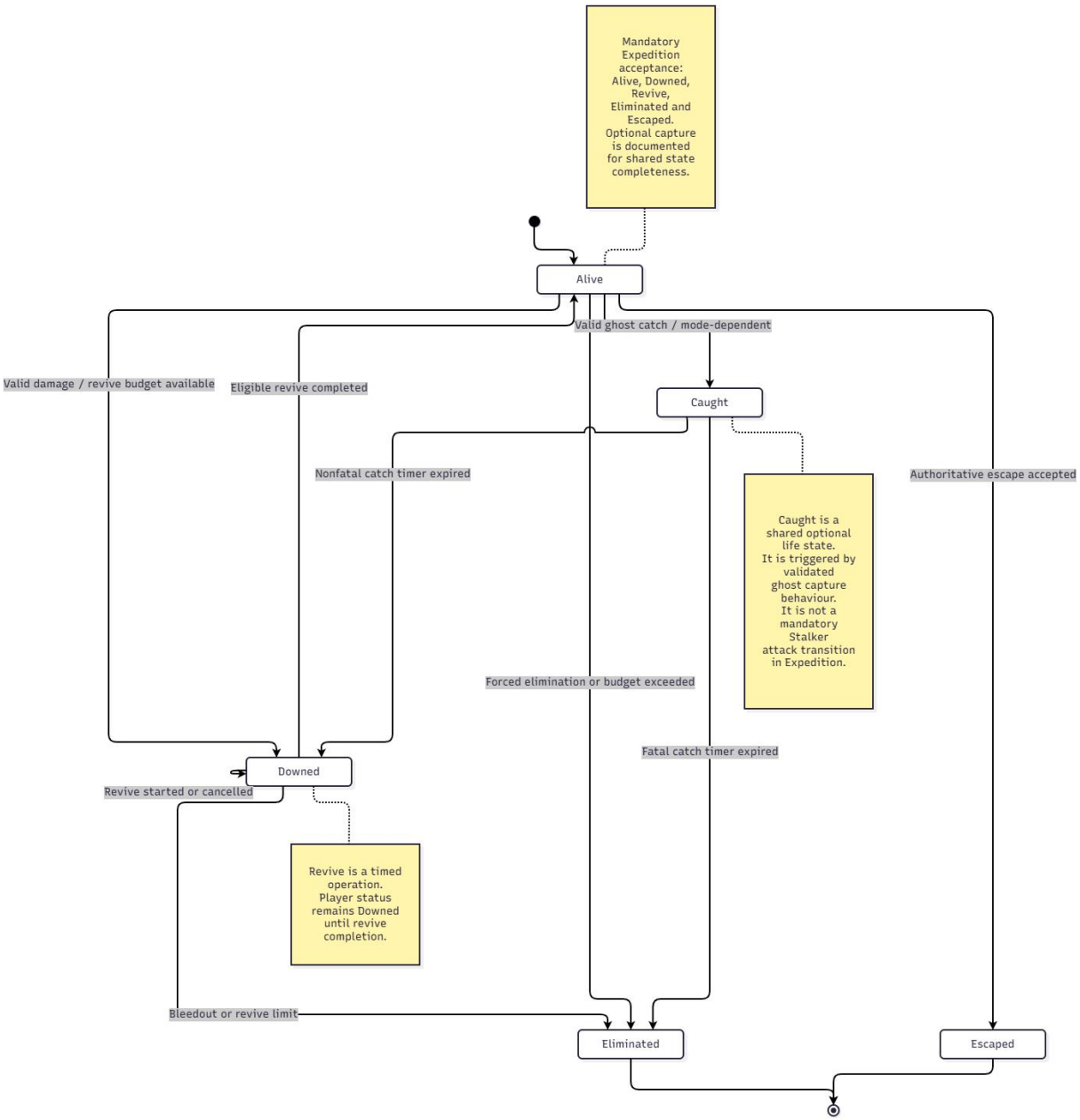


Figure 11 represents the shared authoritative player life-state model implemented by NetworkPlayerLifeState. The mandatory Expedition acceptance path covers Alive, Downed, eligible revive, Eliminated, and Escaped outcomes.

The Caught state is retained for shared, mode-dependent capture behaviour and is not a mandatory Stalker attack transition in Expedition. A validated ghost capture may lead to Downed or Eliminated when the configured capture timer expires.

Revive is handled as a timed authoritative operation while the player's status remains Downed. Successful completion returns the player to Alive, subject to the applicable revive conditions and equipment requirements.

This distinction preserves the shared implementation model without extending the mandatory Expedition gameplay scope or introducing an additional Stalker capture requirement.

Figure 11. Player life-state transitions (abstracted from NetworkPlayerLifeStatus)

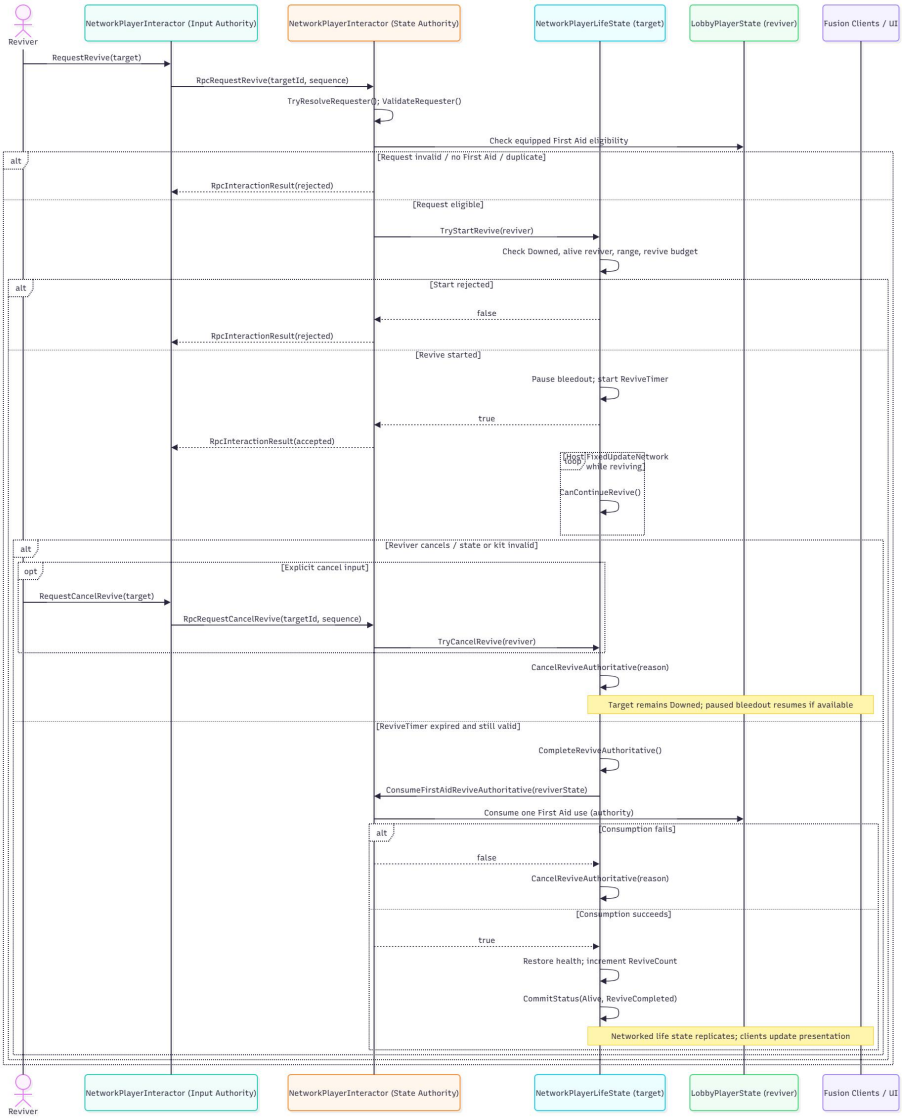


Figure 12. Eligible teammate revive sequence

3.4 Expedition Mission and Objective Control

The Expedition mission is controlled by the Host-authoritative NetworkMatchState. The active top-level progression follows CoreObjective, Zone2Objective, Zone3FindFrigate, Zone3PushFrigate, FinalHunt, and Escape, with MatchEnded representing the terminal match phase.

Zone 2 uses nested Zone2MissionStage values to control security terminal discovery, relay repair, Security Hold execution, authorization-code generation, and zone access. The transition from Zone2Objective to Zone3FindFrigate occurs only after the authoritative system accepts the valid zone access code. Zone2Completed is recorded as part of this successful transition.

Security Hold can return to a ready state when no valid operators remain. Relay repair can also be reset when the configured retry window expires. These transitions are controlled by the authoritative match system rather than client-side state changes.

The NetworkMatchPhase enumeration retains Puzzle and SecurityHold values for legacy compatibility. They are not standalone top-level phases in the active mandatory Expedition route. In particular, the legacy puzzle completion method does not advance the current mission. Security Hold is implemented through Zone2MissionStage.SecurityHold within Zone2Objective.

Figure 13 presents the primary authoritative Expedition state progression, including the active Zone 2 substates. Match-ending conditions are evaluated separately and may terminate an active mission with the appropriate Win or Lose result.

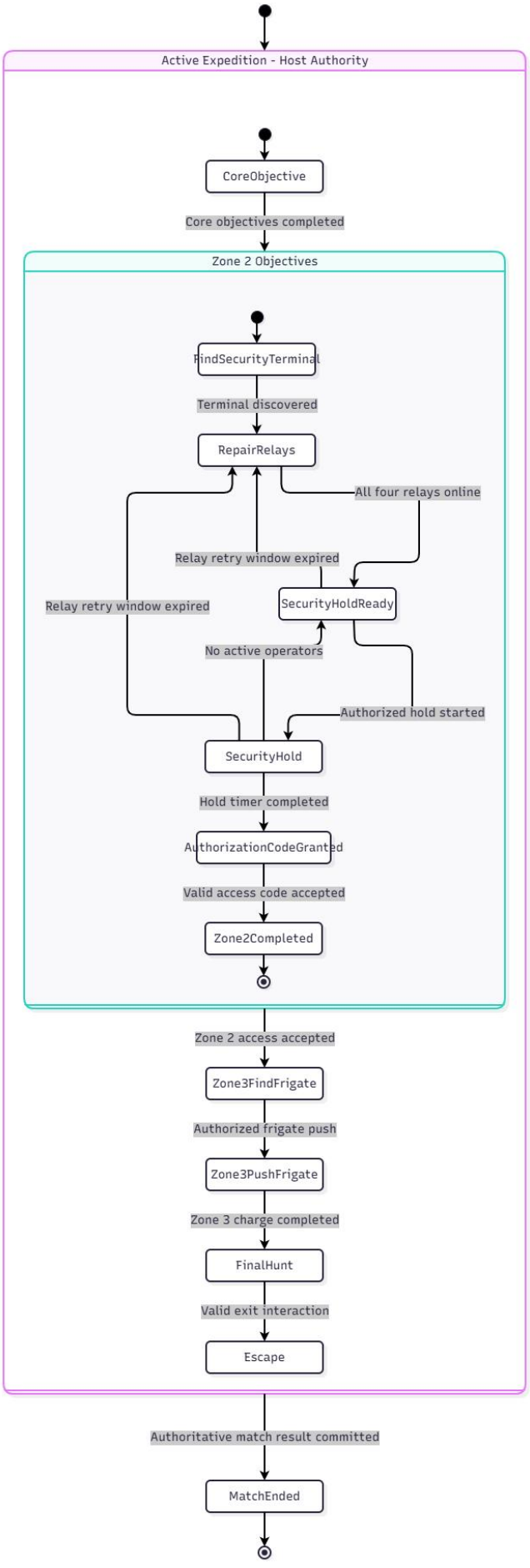

Active Expedition - Host Authority
CoreObjective
Core objectives completed
Zone 2 Objectives
FindSecurityTerminal
Terminal discovered
RepairRelays
All four relays online
Relay retry window expired
Relay retry window expired
SecurityHoldReady
No active operators
Authorized hold started
SecurityHold
Hold timer completed
AuthorizationCodeGranted
Valid access code accepted
Zone2Completed
Zone 2 access accepted
Zone3FindFrigate
Authorized frigate push
Zone3PushFrigate
Zone 3 charge completed
FinalHunt
Valid exit interaction
Escape
Authoritative match result committed
MatchEnded

Figure 13. Primary Expedition phase progression (successful path)

Figure 13 Notes

Zone 2 completion and the transition to Zone 3 are committed through one authoritative operation after successful access-code validation. A valid exit interaction may also advance the mission phase and resolve the match within the same operation.

MatchEnded is the terminal phase, while Win and Lose are recorded as separate match results. Timeout handling may still produce a Win result when a player has already escaped.

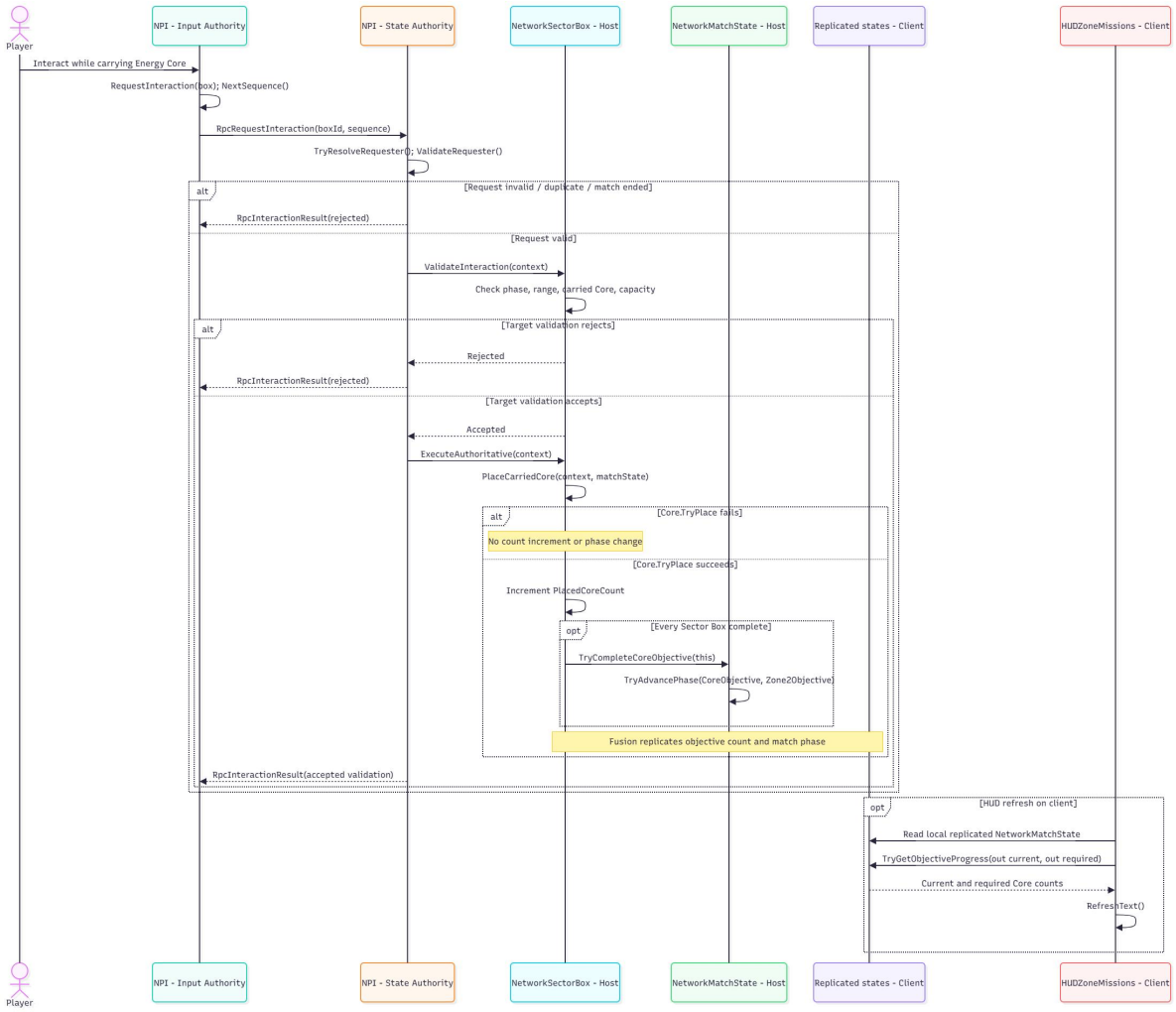


Figure 14. Authoritative objective interaction and HUD update sequence

Class / Component	Design responsibility / selected operations
NetworkPlayerInteractor	Routes interaction requests as RPCs to the owning State Authority, which validates them.
NetworkSectorBox	Validates Energy Core insertion against authoritative state and updates accepted objective progress.
NetworkMatchState	Owns replicated mission phase, objective flags and match result snapshots.
PlayerInteraction / IInteractable	Selects eligible local targets and displays interaction prompts; cannot directly commit shared state.
Zone2MissionDirector / Zone3MissionDirector	Manages mission prerequisites and scene presentation, using Host-accepted game state.

3.5 Stalker Monster AI

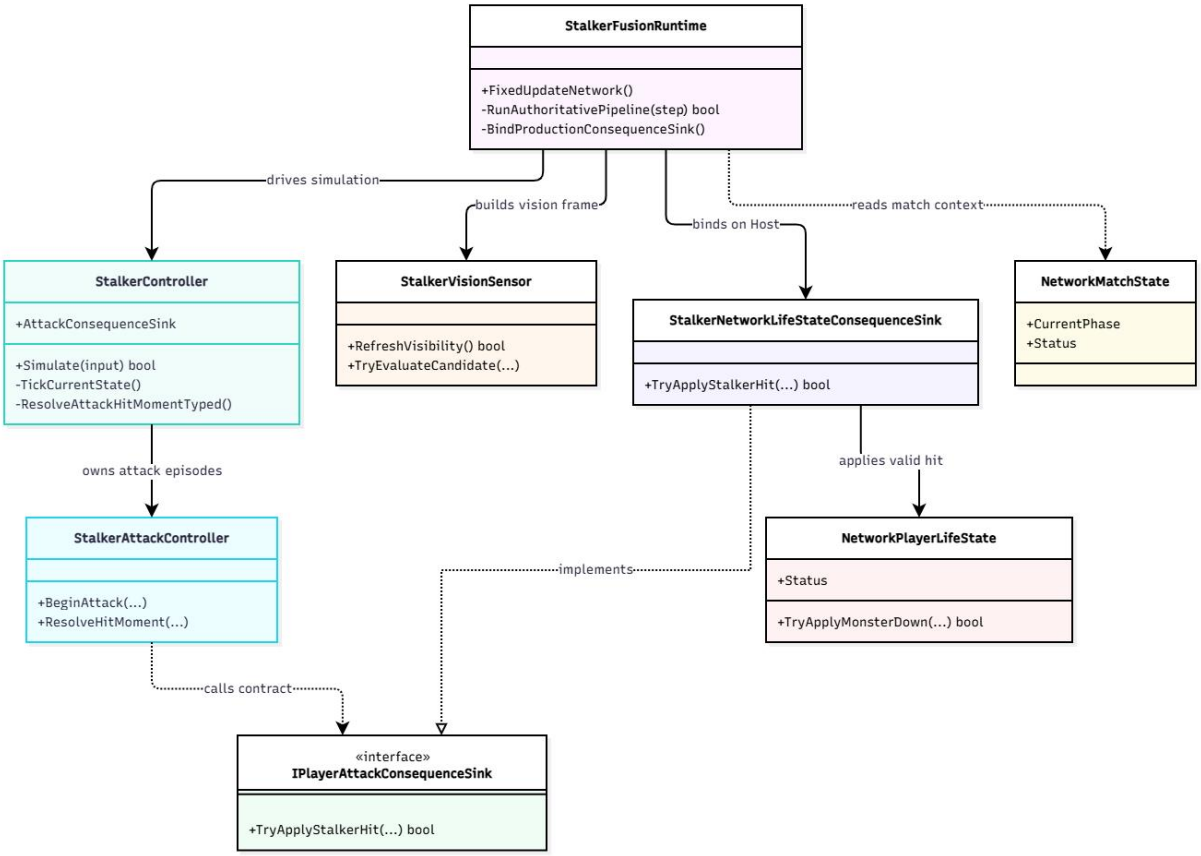


Figure 15. Stalker AI class/component diagram

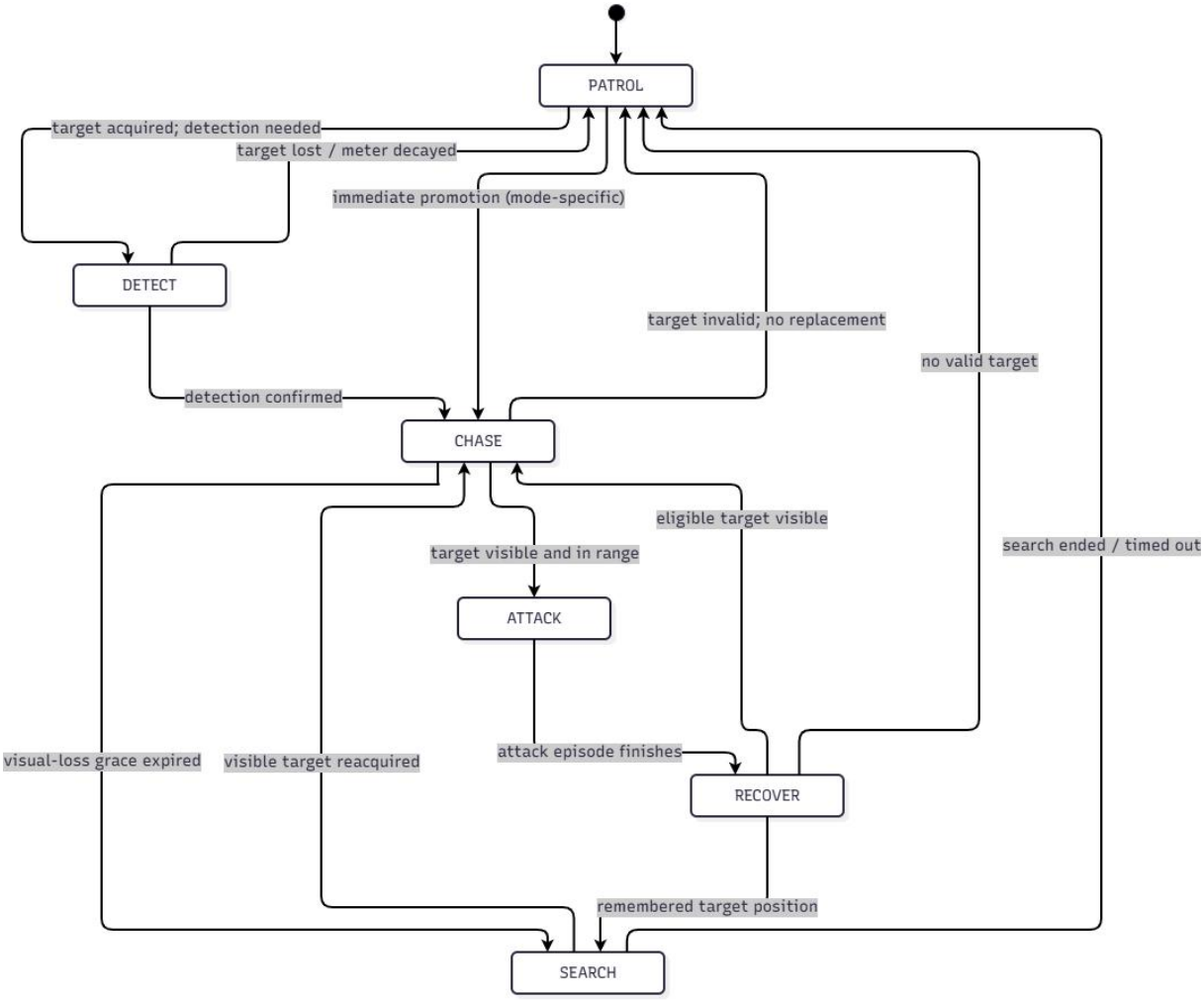


Figure 16. Primary Stalker finite-state machine (logical transitions)

StalkerState enum includes PATROL, DETECT, CHASE, ATTACK, RECOVER and SEARCH. The diagram expresses expected high-level transitions; it does not assert every runtime transition must traverse the same path. Exact vision angle, range, noise scoring, UV resource and cooldown are controlled by serialized/game settings and require scene tests.

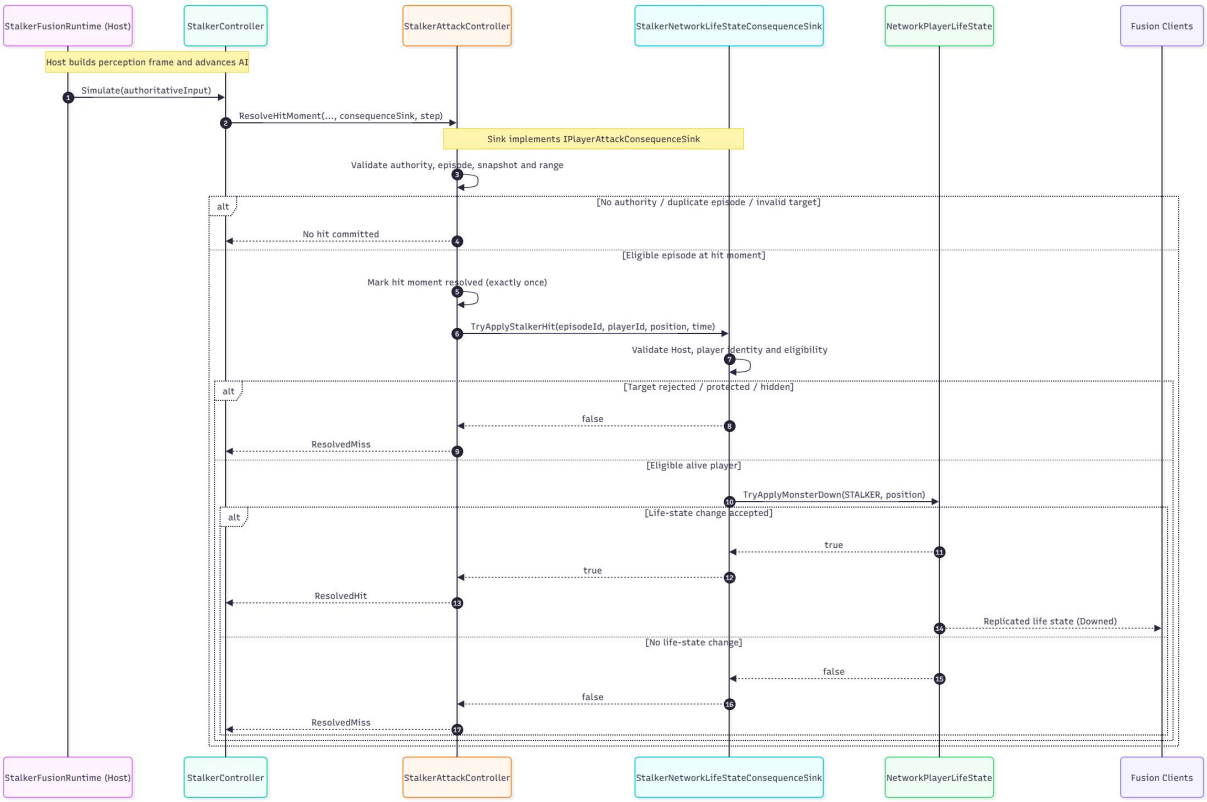


Figure 17. Host-authoritative Stalker attack and replicated outcome sequence

3.6 Adaptive Experience Director and Telemetry

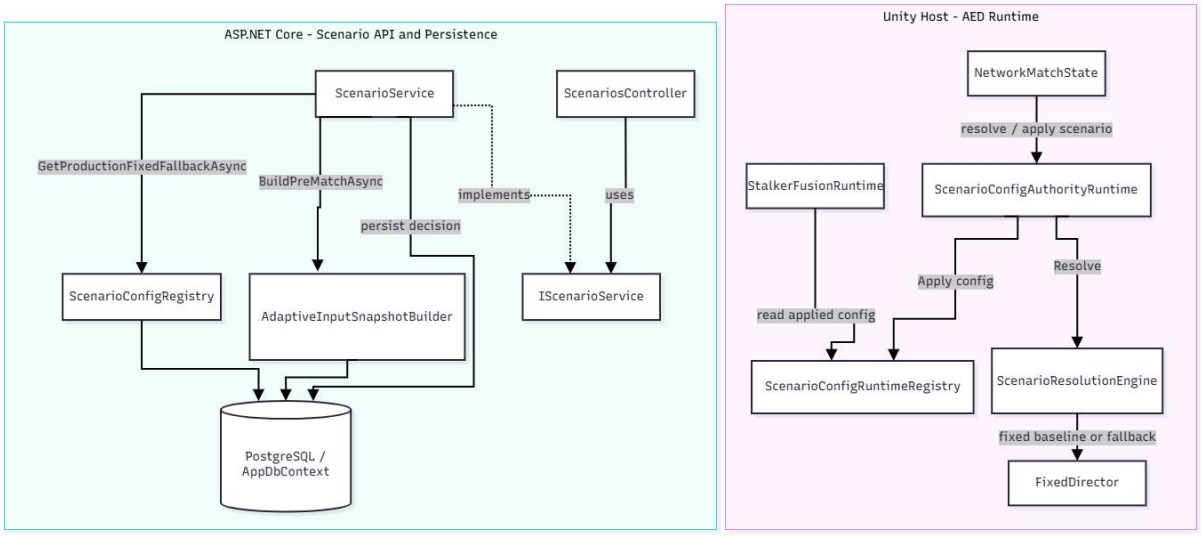


Figure 18. AED and pre-match scenario decision components

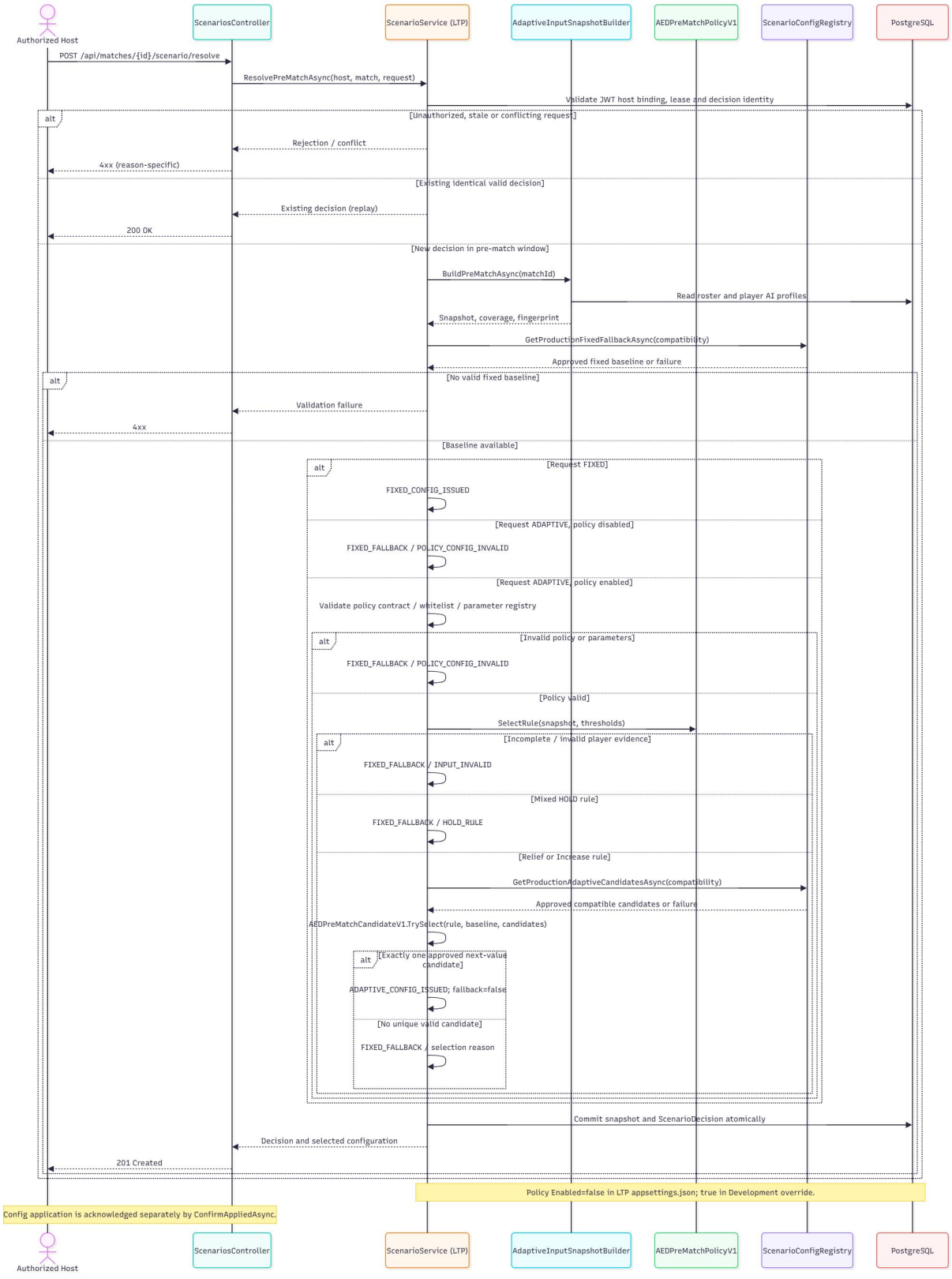


Figure 19. Pre-match scenario resolution with bounded Adaptive selection and Fixed Fallback

ScenarioService loads an approved fixed baseline for every resolution. When Adaptive mode is requested and AEDPreMatchPolicy is enabled, it validates the policy contract and observed Survival/Noise team evidence, then selects an allowed configuration within bounded support-item/chase-speed candidates. Missing evidence, invalid policy or unavailable candidates lead to a fixed fallback with a recorded reason. The decision, fingerprint and applied configuration are persisted for traceability; in-match pilot extensions remain separately gated.

3.7 Match Result and Reward Processing

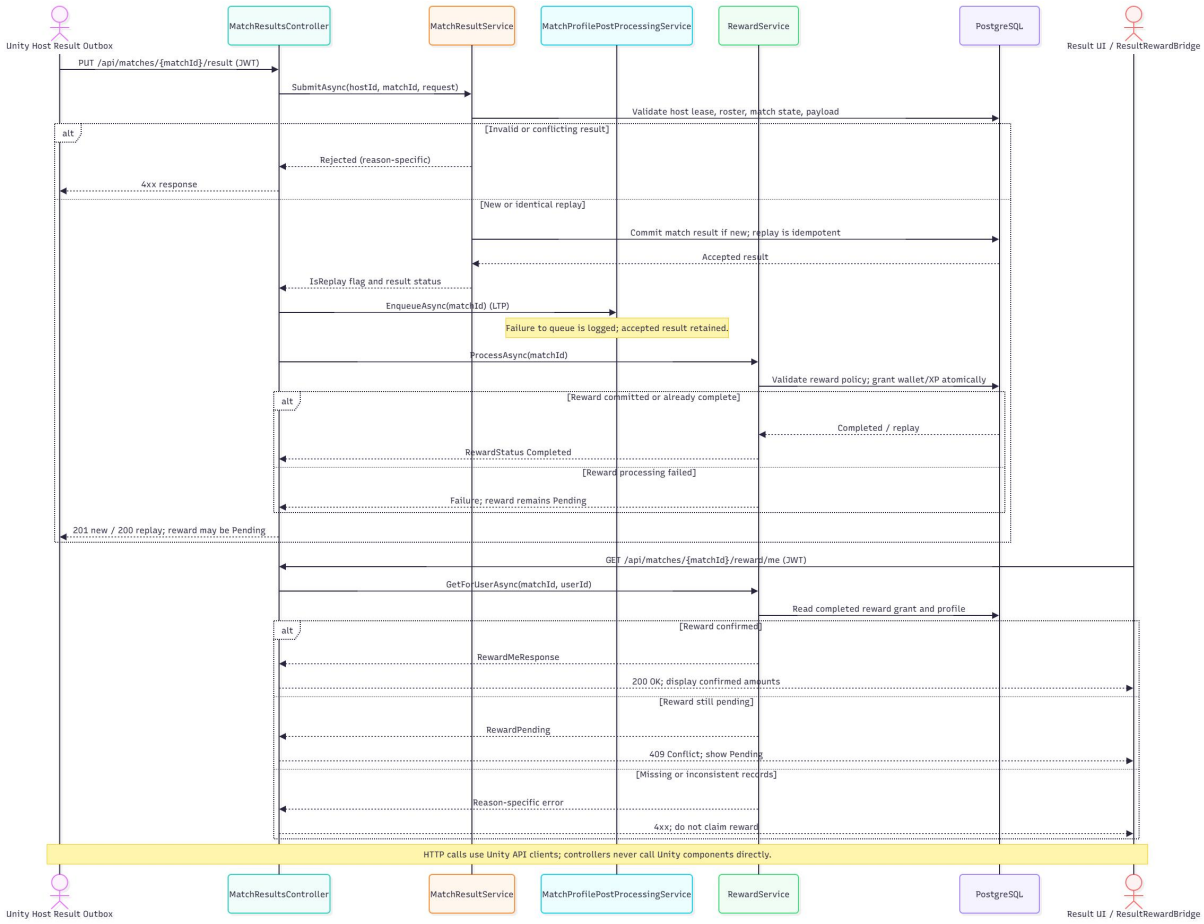


Figure 20. Match result validation, replay handling and reward-processing flow

Class / Component	Design responsibility / selected operations
MatchResultsController	Receives PUT /api/matches/{matchId}/result with authenticated user.
MatchResultService	Checks Host binding, match status, lease, roster, payload and idempotent replay.
RewardService	Applies configured reward/progression policies and stores one set of grants.
ResultRewardBridge	Loads confirmed player rewards through the backend API and distinguishes confirmed, pending and failed reward states.

3.8 Virtual Shop, Wallet and Inventory

Field	Specification
Purpose	Buy eligible items using persistent ECHO Credits, grant ownership and maintain an auditable ledger.
Inputs / triggers	Authenticated UserId, ShopItemId and IdempotencyKey.
Authority / owner	Backend PurchaseService and PostgreSQL transaction; Unity only requests and renders server-accepted state.
Failure / negative path	Inactive/unavailable item, already owned, insufficient balance, conflicting key, concurrency and unauthorized requests never charge twice.

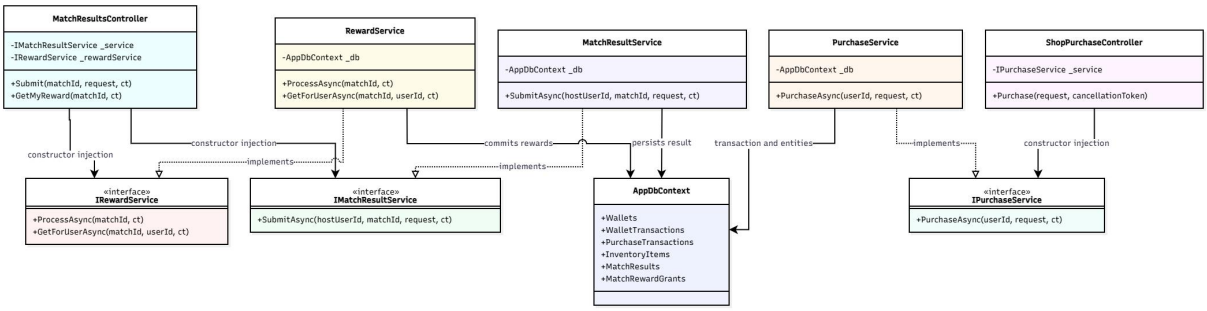


Figure 21. Backend economy and result service dependencies

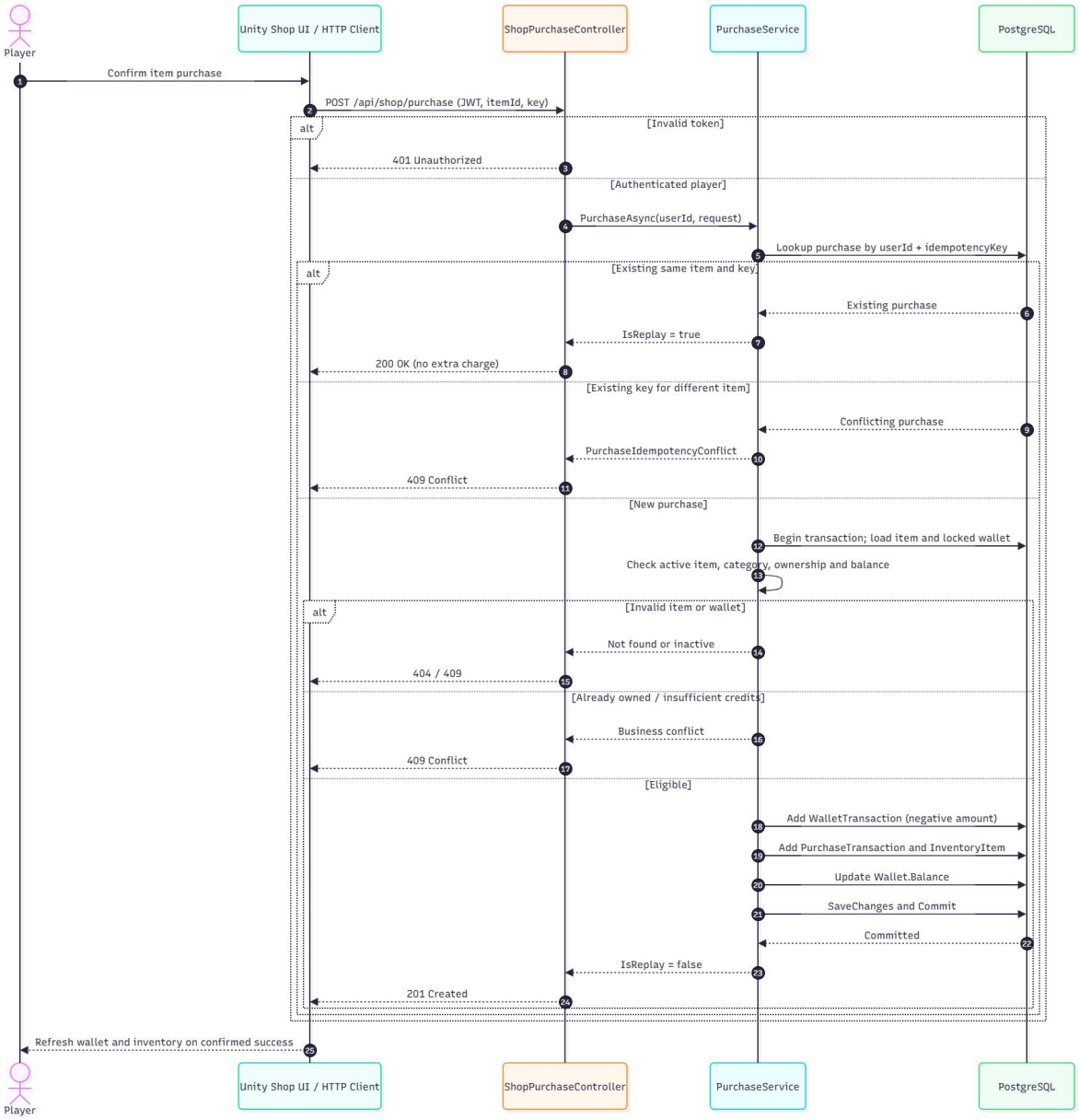


Figure 22. ECHO Credits purchase with atomic updates and replay safety

Class / Component	Design responsibility / selected operations
ShopPurchaseController	Protected POST /api/shop/purchase endpoint.
PurchaseService	Validates price/availability/ownership/balance and handles replay.
Wallet / WalletTransaction	Balance plus immutable debit/grant history.
PurchaseTransaction / InventoryItem	Persistent purchase identity and granted ownership.
InventoryService / LoadoutService	Reads and validates owned/compatible equipment selection.

The registered economy uses ECHO Credits only: validated match rewards or authorized administrative/test allocations. Real-money checkout and payOS are outside the acceptance scope; the limited Admin UI supports permitted, audited operations.

3.9 UI, HUD and Gameplay Feedback

Field	Specification
Purpose	Display mission status, equipment, interaction prompt, teammate life and authoritative result without inventing accepted state.
Inputs / triggers	NetworkMatchState, PlayerLifeState, scene interactions, backend reward/shop responses.
Authority / owner	Local UI renders replicated gameplay state or authenticated API response; server/Host owns accepted mutations.
Failure / negative path	Hide unsupported/inapplicable prompts; show rejected actions, disconnected status, pending reward and session failure correctly.
SRS trace	FR-UI-01–05

Class / Component	Design responsibility / selected operations
Main menu / Lobby UI	Show authorized navigation, selected room/mode/difficulty, ready state and failure feedback.
PlayerInteraction.PromptChanged	Contextual interaction message based on current eligible target.
Match/Player HUD presenters	Read networked objective, status and tool state; update only on actual transitions.
ResultSession / ResultRewardBridge	Distinguish local match outcome from accepted backend reward.
Admin UI	Shows records permitted by backend role; no direct database modification.

Design rule: HUD should be event/state-driven rather than showing all objectives or warnings continuously. Scene-specific art, animation and sound clip lists belong to the GDD/asset inventory; this SDD documents their triggering contracts rather than replicating the full content catalogue.

3.10 Main Business Workflows

Four end-to-end workflows cover authentication and lobby admission, Expedition progression, Stalker encounters and player survival, and result/reward with the virtual economy. All workflows use the selected 1–4-player Expedition scope and Host-authoritative state changes. The diagrams describe design paths; test execution is documented separately in Report 5.

3.10.1 WF-01 – Authentication, Legal Agreement and Lobby

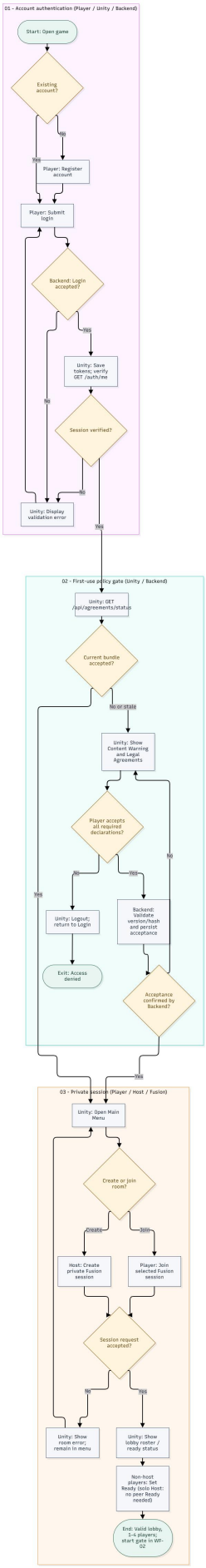

01 - Account authentication (Player / Unity / Backend)
Start: Open game
Existing account?
No
Yes
Player: Register account
Player: Submit login
Backend: Login accepted?
Yes
Unity: Save tokens; verify GET /auth/me
No
Session verified?
No
Unity: Display validation error
Yes
02 - First-use policy gate (Unity / Backend)
Unity: GET /api/agreements/status
Current bundle accepted?
No or stale
Unity: Show Content Warning and Legal Agreements
Player accepts all required declarations?
No
Yes
Unity: Logout; return to Login
Exit: Access denied
Backend: Validate version/hash and persist acceptance
Acceptance confirmed by Backend?
No
Yes
Yes
03 - Private session (Player / Host / Fusion)
Unity: Open Main Menu
Create or join room?
Create
Join
Host: Create private Fusion session
Player: Join selected Fusion session
Session request accepted?
No
Yes
Unity: Show room error; remain in menu
Unity: Show lobby roster / ready status
Non-host players: Set Ready (solo Host: no peer Ready needed)
End: Valid lobby, 1–4 players; start gate in WF-02

Figure 23. Authentication, Legal Agreement and Lobby Workflow

Field	Description
Owner	Hứa Thành Đạt
Actors / Components	Guest, Player, Host Player, Unity Client, Backend API, Photon Fusion
Trigger	A user registers or logs in and requests access to a private session.
Business Outcome	A user with a verified session and current legal acceptance enters a valid 1–4-player lobby; solo Host is supported.
State Changes	Unauthenticated → Authenticated; Agreement Missing/Outdated → Accepted; Not Joined → In Lobby / Ready.
Related FRs	FR-AUT-01–05; FR-MUL-01–03; FR-UI-01.
Related Test Cases	TC-WF01-01 Valid login/lobby; TC-WF01-02 first-use legal acceptance; TC-WF01-03 invalid credentials, declined agreement or room.

3.10.2 WF-02 – Expedition Gameplay and Match Outcome

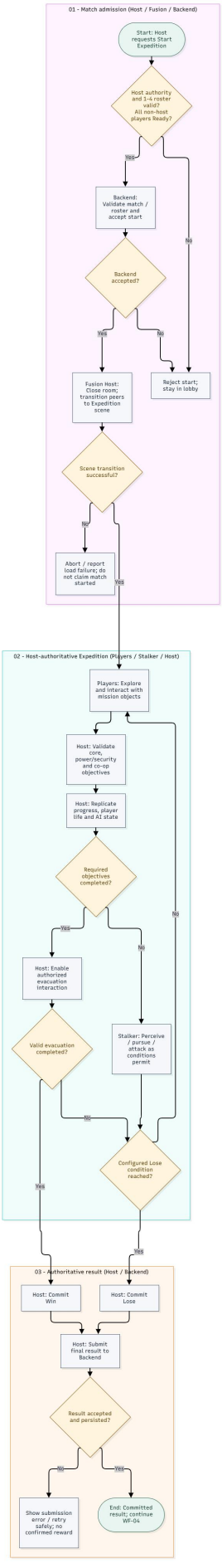

01 - Match admission (Host / Fusion / Backend)
Start: Host requests Start Expedition
Host authority and 1-4 roster valid? All non-host players Ready?
Yes
No
Backend: Validate match / roster and accept start
Backend accepted?
Yes
No
Fusion Host: Close room; transition peers to Expedition scene
Reject start; stay in lobby
Scene transition successful?
No
Yes
Abort / report load failure; do not claim match started
02 - Host-authoritative Expedition (Players / Stalker / Host)
Players: Explore and interact with mission objects
Host: Validate core, power/security and co-op objectives
Host: Replicate progress, player life and AI state
Required objectives completed?
Yes
No
No
Host: Enable authorized evacuation interaction
Valid evacuation completed?
Stalker: Perceive / pursue / attack as conditions permit
No
Configured Lose condition reached?
Yes
Yes
03 - Authoritative result (Host / Backend)
Host: Commit Win
Host: Commit Lose
Host: Submit final result to Backend
Result accepted and persisted?
No
Yes
Show submission error / retry safely; no confirmed reward
End: Committed result; continue WF-04

Figure 24. Expedition Gameplay and Match Outcome Workflow

Field	Description
Owner	Đặng Huy Hoàng
Actors / Components	Host Player, Players, Unity Client, Photon Fusion, Stalker AI, Backend API
Trigger	A Host requests to start Expedition from a valid lobby.
Business Outcome	Host validates match start and objectives, synchronizes Win/Lose, and submits an authorized result for Backend validation.
State Changes	Lobby → In Match → Match Ended; Objective Phases → Escape/Failure; Result Not Committed → Win/Lose Committed.
Related FRs	FR-MUL-03–05; FR-PLY-01–05; FR-EXP-01–05; FR-AI-01–05; FR-UI-02–03.
Related Test Cases	TC-WF02-01 successful Expedition; TC-WF02-02 configured Lose; TC-WF02-03 invalid start, premature evacuation or scene failure.

3.10.3 WF-03 – Stalker AI Gameplay and Player Survival

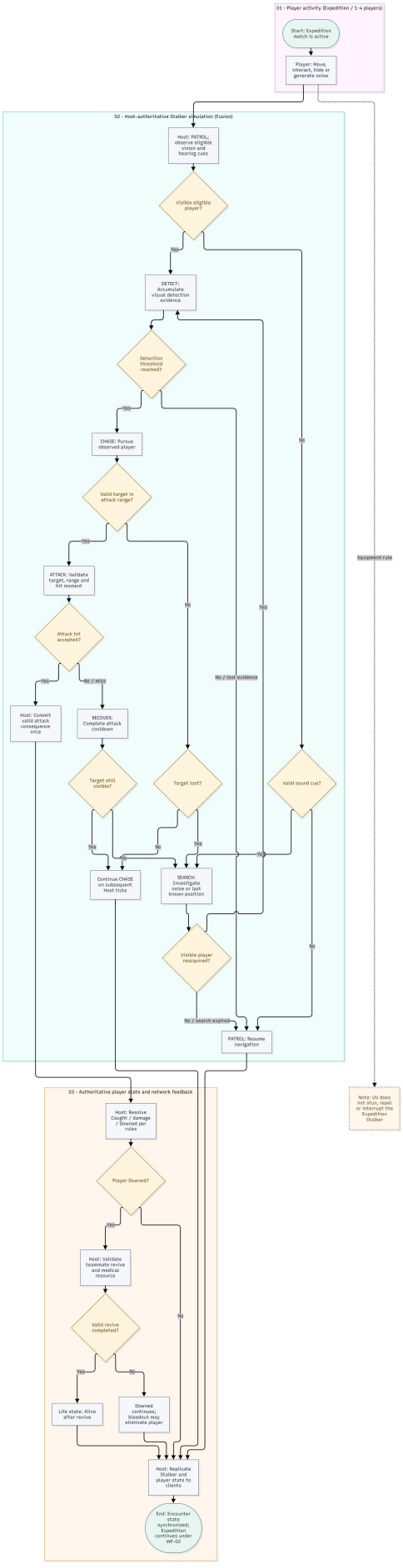
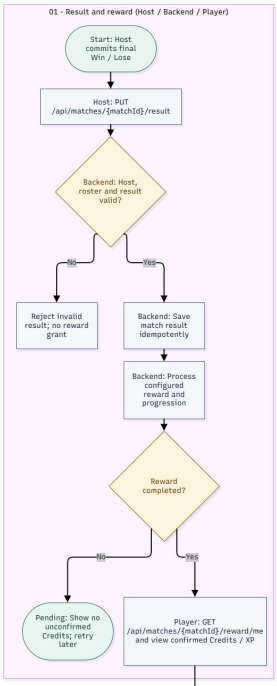

01 - Player activity (Expedition / 1-4 players)
Start: Expedition match is active
Player: Move, interact, hide or generate noise
02 - Host-authoritative Stalker simulation (Fusion)
Host: PATROL: observe eligible vision and hearing cues
Visible eligible player?
Yes
DETECT: Accumulate visual detection evidence
Detection threshold reached?
Yes
CHASE: Pursue observed player
Valid target in attack range?
Yes
ATTACK: Validate target, range and hit moment
Attack hit accepted?
Yes
No / miss
Host: Commit valid attack consequence once
RECOVER: Complete attack cooldown
Target still visible?
Yes
No
Continue CHASE on subsequent Host ticks
Target lost?
Yes
No
SEARCH: Investigate noise or last known position
Visible player reacquired?
Yes
No / search expires
PATROL: Resume navigation
No
No / lost evidence
Yes
Valid sound cue?
Yes
No
Equipment rule
Note: UV does not stun, repel or interrupt the Expedition Stalker
03 - Authoritative player state and network feedback
Host: Resolve Caught / damage / Downed per rules
Player Downed?
Yes
Host: Validate teammate revive and medical resource
Valid revive completed?
Yes
No
Life state: Alive after revive
Downed continues; bleedout may eliminate player
No
Host: Replicate Stalker and player state to clients
End: Encounter state synchronized; Expedition continues under WF-02

Figure 25. Stalker AI Gameplay and Player Survival Workflow

Field	Description
Owner	Lâm Thanh Phúc
Actors / Components	Player, Teammate, Host / State Authority, Stalker AI Runtime, Photon Fusion clients
Trigger	An active Expedition player becomes a valid visible target or produces an eligible noise cue.
Business Outcome	Host evaluates Stalker perception, pursuit/search and validated attacks. Player consequences and eligible revive outcomes are replicated to all clients.
State Changes	Stalker: PATROL, DETECT, CHASE, ATTACK, RECOVER, and SEARCH under authoritative AI transitions. Mandatory Expedition player outcomes: Alive → Downed → Alive (successful revive) or Eliminated; Alive → Escaped when the authoritative evacuation condition is satisfied. Caught is a shared, mode-dependent player state and is not a required transition in the mandatory Stalker encounter workflow.
Related FRs	FR-AI-01–05; FR-PLY-01–04; FR-MUL-04; FR-UI-02.
Related Figures	Figures 11–12 and 15–17 (life state, revive, Stalker components/FSM/attack).
Related Test Cases	TC-WF03-01 sight/detection; TC-WF03-02 noise/search; TC-WF03-03 authoritative attack; TC-WF03-04 revive; TC-WF03-05 UV does not interrupt Stalker.

Scope note: UV does not stun, repel or interrupt the Expedition Stalker. Adaptive AI remains documented separately in Section 3.6 and Figures 18–19.

3.10.4 WF-04 – Match Result, Reward and Virtual Economy


01 - Result and reward (Host / Backend / Player)
Start: Host commits final Win / Lose
Host: PUT /api/matches/{matchId}/result
Backend: Host, roster and result valid?
No
Yes
Reject invalid result; no reward grant
Backend: Save match result idempotently
Backend: Process configured reward and progression
Reward completed?
No
Yes
Pending: Show no unconfirmed Credits; retry later
Player: GET /api/matches/{matchId}/reward/me and view confirmed Credits / XP

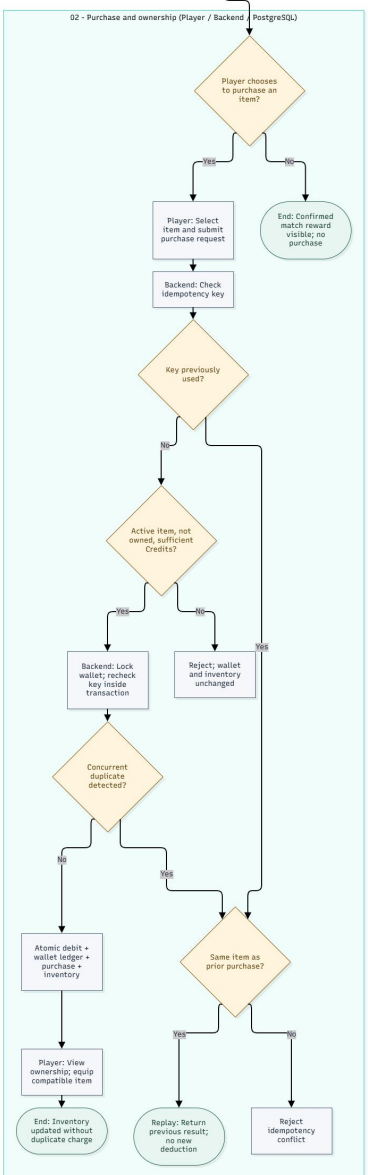

02 - Purchase and ownership (Player / Backend / PostgreSQL)
Player chooses to purchase an item?
Yes
No
Player: Select item and submit purchase request
End: Confirmed match reward visible; no purchase
Backend: Check idempotency key
Key previously used?
No
Yes
Active item, not owned, sufficient Credits?
Yes
No
Backend: Lock wallet; recheck key inside transaction
Reject; wallet and inventory unchanged
Concurrent duplicate detected?
No
Yes
Atomic debit + wallet ledger + purchase + inventory
Same item as prior purchase?
Yes
No
Player: View ownership; equip compatible item
End: Inventory updated without duplicate charge
Replay: Return previous result; no new deduction
Reject idempotency conflict

Figure 26. Match Result, Reward and Virtual Economy Workflow

Field	Description
Owner	Nguyễn Nhật Lệ Thuyên
Actors / Components	Host Player, Player, Unity Client, Backend API, PostgreSQL
Trigger	Host submits a finalized Win/Lose result; a player may subsequently choose to purchase an item.
Business Outcome	Backend validates and persists results/rewards without duplication. Optional valid purchases debit Credits and grant ownership atomically.
State Changes	Result Not Stored → Stored; Reward Pending → Completed on confirmation; Wallet balance updated; unowned item → owned after purchase.
Related FRs	FR-ECO-01–05; FR-UI-03–04.
Related Test Cases	TC-WF04-01 reward and purchase; TC-WF04-02 pending reward/replay; TC-WF04-03 insufficient Credits and duplicate/idempotency conflict.

Verification boundary: TC-WF01–TC-WF04 identifiers are proposed Report 5 references, not claims of executed or passed tests. Adaptive Candidate selection is not a prerequisite for the Stalker encounter workflow.

4. Art Direction and Moodboard (Game Projects)

This section documents visual direction against the current GitHub main source snapshot 02ada8537795 (committed 02 November 2026 UTC), supplemented by the project GDD images and Unity assets. The technical architecture, diagrams and workflow acceptance in Sections 1–3 remain referenced to the Review 2 LTP design baseline 08de8d2347d1. Source presence proves a resource or code path exists; it does not replace visual inspection of a final Windows build. The mandatory visual scope is the Research Facility Expedition; Purge is supplementary.

4.1 Visual Summary and Pillars

Item	Description verified or adopted
Setting	One mandatory Research Facility Expedition with three connected objectives/zones; optional Purge environment treated separately
Visual style	Dark industrial sci-fi horror with distressed title graphics, muted interface surfaces, red signals and functional low-light highlights
Camera	First-person Windows client; Camera/graphics settings are runtime-configurable and cannot be reduced to one fixed field of view
Target device	Windows 10/11 PC, mouse and keyboard; maintain readability at chosen game resolution and GPU-saver quality mode
Three keywords	Industrial — Uneasy — Legible
Visual sources	Main Menu and SciFi scenes in main; IndustrialUiStyle, UiTypographyTheme, HUDPresentationStyle and MainMenuTypographyController; earlier GDD screenshot references

Visual pillar	Application in ECHO PROTOCOL	Verification
Readable survival information	Active objectives, interactables, teammate conditions and critical danger must be visually distinguishable	Check screenshots and recorded UI interactions at test resolution
Environmental tension	Industrial machinery, damaged systems, dark corridors and restricted sightlines create suspense	Review Zone 1–3 lighting and navigation screenshots
Consistent system language	Power, terminal, relay, equipment, threat and result cues reuse controlled color/icon conventions	Compare HUD, mission props and UI against the same style sheet
Performance awareness	Reuse materials, limit costly full-screen effects and excessive particles, avoid visually noisy HUD	Profile the Windows build and the documented graphics preset

4.2 Overall Mood

The intended progression moves from isolation and uncertainty in the facility lobby to investigation, electrical-system tension and the final escape. On main, the menu presentation uses deliberately distressed industrial typography; the gameplay environment remains dark without making controls or objectives unreadable. Threat cues must show through meaningful HUD and world changes, not endless full-screen effects. The Stalker is survived through navigation and cooperation: UV is not a stun or repel mechanic for the Expedition Stalker.

We are	We are NOT
Low-light sci-fi industrial horror	Cartoon comedy or exaggerated neon action
Contrast focused on objectives and threats	Uniformly dark screens that obscure necessary interactions
Contextual camera and damage effects	Constant full-screen flashes/shake during normal movement
Readable iconography and state-aware HUD	Permanently crowded overlays unrelated to the current mission



Art Direction Figure AD-01. Existing GDD Main Menu capture (October 2026 design evidence); the current main MainMenu scene and typography must be re-captured for final acceptance screenshots.

4.3 Color Palette and Readability Rules

The table below uses actual Color32 values declared in main/IndustrialUiStyle.cs, not an invented global palette. These values are code definitions for functional Main Menu/Lobby surfaces; scene lighting and texture colours are authored separately. The Main Menu typography additionally uses near-white #F8F7F4 for its title and red #CD1A26 for its signal subtitle. HUDPresentationStyle adds warning #D8A85E and danger #D0685F, so one universal red/teal palette must not be claimed.

Role (main source)	Verified HEX	Use / source
UI foreground text	#E5E3DA	IndustrialUiStyle.Text; normal readable text
Muted information	#ADB1B0	IndustrialUiStyle.Muted; lower-emphasis labels
Interface accent	#C9BF8C	IndustrialUiStyle.Accent; selection and restrained highlights
Primary/blood red	#5B141A	IndustrialUiStyle.Blood; strong primary surface emphasis
Dark panel	#090D0E	IndustrialUiStyle.Panel; alpha 245/255 in source
Panel border	#5E5B57	IndustrialUiStyle.Border; alpha 185/255 in source

Readability: accepted success, invalid input and pending states must use both text and a distinct icon/shape, never colour alone. The main HUD source uses a 1600 × 900 Canvas reference resolution where scale-with-screen-size applies; test actual Windows scaling at the team's benchmark resolution. Avoid applying main-menu red to all game systems; HUD warning and health/damage conventions are defined separately in HUDPresentationStyle. Final contrast and accessibility compliance require screenshots from the accepted build.

4.4 Lighting and Atmosphere

The current main SciFi scene and GameGraphicsSettings.cs govern the gameplay presentation, including graphics quality, VSync, display mode and an optional GPU Saver/URP performance setting. Zone 1 uses low-key machinery and recoverable energy-core landmarks; Zone 2 emphasizes relay/terminal/security feedback; Zone 3 raises urgency around frigate and evacuation. A valid UV/light effect must not visually imply that it interrupts the Expedition Stalker. Do not treat installed post-processing packages or prefab materials as proof that every listed fog, vignette or bloom effect is enabled in the actual scene.

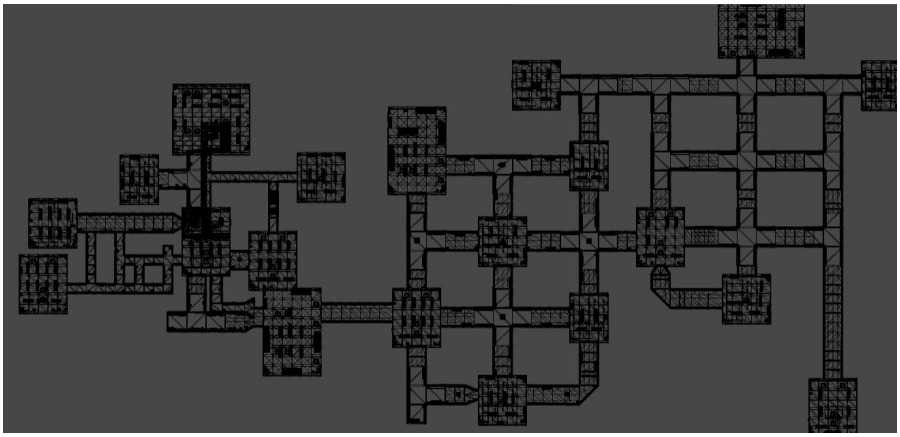
Verification: compare MainMenu.unity and SciFi.unity at main 02ada8537795 against the captured Windows build. Check objective silhouettes, flashlight readability, HUD contrast, graphics presets and whether scene volumes/lighting have measurable performance impact. Do not claim exact per-scene light intensities or post-processing cost without Unity Inspector/Profiler evidence.

4.5 Environments and Locations

Location	Theme	Main materials / objects	Landmark / gameplay connection
Research Facility —	Entry/recovery	Metal corridors, machinery,	Energy core handling and core

Location	Theme	Main materials / objects	Landmark / gameplay connection
Zone 1		energy cores and power hubs	objective progression
Research Facility — Zone 2	Security and electrical systems	Terminal panels, relays, access doors, alarms	Security terminal, relay repair, Security Hold and access authorization
Research Facility — Zone 3	Final industrial passage / escape	Frigate-related machinery, charge/push interfaces and exit path	Frigate sequence, Final Hunt and evacuation
Flooded residential Purge environment	Damaged submerged neighbourhood	Flooded urban props, rusty burn barrel and metal grate	Supplementary game mode; outside mandatory Expedition acceptance

The three mandatory zones are gameplay stages of one Research Facility Expedition, not three required map products. Main contains the current SciFi scene and associated facility assets; the GDD floorplan and objective reference images provide an earlier documentary view and are not automatically guaranteed pixel-identical to the latest scene. Main also contains Scene_A and Scene_B/Purge content, which must remain clearly marked as implemented extensions outside mandatory Expedition acceptance.



Art Direction Figure AD-02. Research Facility layout from the existing GDD; annotate and reconfirm objective landmarks against main/SciFi.unity before final publication.

4.6 Characters and Factions

The primary Expedition threat is the Stalker, distinct from supplementary Lizard/Fishcat and other Purge entities. Main contains a dedicated Stalker controller and presentation layer; the figure below is a project-provided Stalker visual reference, not evidence of the current render at every graphics preset. Player appearance/customization and companion previews also exist as resources, but they must not imply an unverified gameplay-stat advantage or a mandatory pet system. In all character states, enemy sightlines, silhouettes and directional feedback must remain legible in low light.

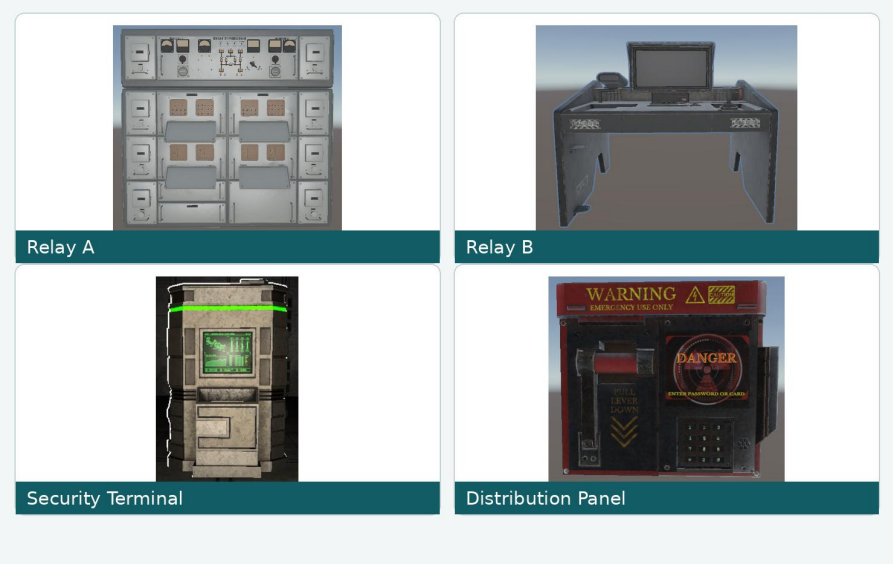


Art Direction Figure AD-03. Stalker screenshot from the pre-existing GDD; confirm with an updated main-branch build capture if submission evidence must match main.

4.7 Items, Props and Objects

Group	Visual identity	Functional reading
Energy cores	Distinct carryable sci-fi resource	Player can identify pickup, carry and deposit state
First Aid	Medical assistance equipment	Downed teammates and valid revive interactions should be understandable
Flashlight and UV	Different equipment functions and UI feedback	UV status does not communicate Stalker stun in Expedition
Power/security systems	Core hub, relays, terminal, doors and alarms	Inactive, active, denied and completed states are distinct
Noise Maker / team tools	Throwable/deployable utility with observable feedback	Tools obey configuration and Host validation
Shop/inventory items	Current catalogue icon and equip ownership state	Price/ownership is backend-authoritative
Rusty barrel/grate	Purge ritual/burn landmark	Supplementary content only

Main contains resources for lobby character/outfit previews, inventory/shop imagery, objective props and environmental items. For required Expedition props, show an actual source-backed item state in each screenshot: energy core available/carried/placed, First Aid available/consumed, terminal/relay locked/active/completed, and UV energy use. Resource presence does not prove shop ownership state or network replication. Optional cosmetics, pets and Purge ritual/burn props are extensions rather than required Expedition deliverables.



Art Direction Figure AD-04. GDD Zone 2 relay, terminal and access-panel assets.

4.8 Animation

The main asset tree includes AC_PlayerCharacter.controller with player idle/walk/run/crouch animation clips, and AC_Stalker.controller with runtime presentation by StalkerAnimatorPresenter. Team-tool animation includes AC_CoreStabilizer_Device.controller. The lizard/creep animations belong to optional content and must not be presented as mandatory Stalker animations. Code and Animator assets confirm animation resources, but final timing and blend correctness require Inspector review and PlayMode observation.

System	Source-backed motion / presentation	Trigger / evidence rule
Player	Player_Idle, Player_Walk_Forward, Player_Run_Forward, Player_Crouch_Idle/Walk_Forward	AC_PlayerCharacter.controller; inspect accepted camera/movement state and blend
Player survival	Damage/downed/revive and elimination feedback through life-state and HUD/audio	Only after authoritative life-state transition; do not invent dedicated clips
Stalker	PATROL/DETECT/CHASE/ATTACK/RECOVER/SEARCH animation presentation	AC_Stalker.controller + StalkerAnimatorPresenter; validate event timing against replicated state
Objectives/tools	Door, relay, terminal, frigate and core-stabilizer presentation	Inspect actual scene binding and AC_CoreStabilizer_Device before claiming each clip is played
Camera/UI	Cinematic loading fades, HUD overlays and	CinematicMatchLoadingScreen and

System	Source-backed motion / presentation	Trigger / evidence rule
	local jumpscare camera/FOV effects	PlayerJumpscareController; state-gated and non-blocking

4.9 UI and HUD Style

Main uses IndustrialUiStyle for menu/lobby functional surfaces and UiTypographyTheme for reusable TMP/legacy fonts. MainMenuTypographyController resolves OswaldTMP, JetBrainsMonoTMP and an industrial/Roboto fallback for title, signal subtitle and buttons; the HUD has its own HUDPresentationStyle. CinematicMatchLoadingScreen provides a separate themed fullscreen progress/loading presentation with staged labels and fade-out. These are different UI contexts, so font, subtitle colour and effects must not be described as one undifferentiated theme. Important objectives and interaction prompts appear conditionally, not as permanent overlays.

Verified main UI assets include UI/MainMenuTypography/OswaldTMP.asset, JetBrainsMonoTMP.asset, IndustrialTitleTMP.asset and UI/LobbyReference/LobbySerif.asset. The code-defined menu colours are in Section 4.3. Screenshot figures below originate in the earlier GDD; final figure acceptance should use a new capture of the current main MainMenu/Lobby/SciFi builds if the scene's typography or layout has changed.



Art Direction Figure AD-05. Four-player lobby GDD screenshot (design reference); visual parity with the current main UI must be checked in the Windows build.

4.10 VFX

Effect	Activation / current source	Readability rule	Evidence status
Jumpscare camera/FOV and fade	PlayerJumpscareController on valid local capture/presentation	Temporary and state-gated; avoid permanent occlusion	Code present; visual/runtime test required
Cinematic loading overlays	CinematicMatchLoadingScreen display, fade and progress state	Loading text and stage progress remain readable	Code present; scene/run evidence required
Damage/critical life feedback	HUD/life-state response to valid damage, Downed and elimination	Preserve objective prompts and revive action visibility	Inspect prefab/scene; do not infer from concept
Energy-core/relay/terminal feedback	Authoritatively accepted objective interactions	Match active/denied/completed phase and timing	Verify in SciFi scene
UV equipment presentation	Valid equipment use without effect on Expedition Stalker behaviour	No Stalker stun/interrupt visual promise	Verify UV/Stalker encounter in build
Purge creature/burn VFX	PurgeVfx and supplementary scene objects only	Never imply included in mandatory Expedition	Optional extension; separate validation

VFX requirements are grounded in main presentation code rather than assuming every imported particle or post-processing asset is active. PlayerJumpscareController contains local camera/FOV and fade logic; the loading screen has UI overlays and material-driven atmosphere. The source also includes PurgeVfx, but that is supplementary. Damage vignette, directional hit feedback, object sparks and zone-specific post-processing are only marked implemented after their scene/prefab bindings and visible output have been tested. Stalker UV feedback must not indicate an interrupt or stun in Expedition.

4.11 Key Art and Store Assets

Main includes project-specific lobby/character portrait and UI texture resources (for example Shop/LobbyPortraits and UI/MainMenuTypography), which can inform store tiles and key art. A full final marketing poster, platform-store capsule set or promotional screenshots cannot be claimed complete merely because these assets exist. The hero imagery should emphasize the research facility, cooperative survival and Stalker threat. The current main scene is the authoritative visual source for newly captured submission screenshots; earlier GDD renders remain illustrative historical evidence.

4.12 Reference Sources and Asset Licences

Source / asset	Use in report	Evidence needed before final publication
Existing ECHO PROTOCOL Unity screenshots	UI, environment, hero mood and gameplay examples	Git commit or build identification; team-created screenshot
Research Facility / Sci-Fi environment packs in KLTN/Assets	Environment and material references	Exact asset name, publisher, asset/store URL and applicable license
KLTN/Assets/Fonts/...	UI typography	Font family, copyright/license and where used
KLTN/Assets/Audio/...	Sound/ambience catalogue	Original/generator/source attribution and licensed third-party samples
Flooded Grounds / optional Lizard/Fishcat assets	Supplementary Purge concepts only	Original asset download URL and license; not mandatory game baseline
Referenced horror games (e.g. DEVOUR, PHASMOPHOBIA)	Mood/intensity references	Cite names/official pages; do not reuse logos, textures or soundtrack without permission

Source verification: GitHub main @ 02ada853779588446254c6063637cbbca0a8bf37 for the current Section 4 implementation, and the existing GDD for image references produced before this commit. Imported asset presence is not a usage licence. The team must record publisher/author, URL, asset version, applicable licence and redistribution conditions for third-party environment packs, textures, audio and fonts; no unverified licence is asserted in this report.

5. Sound Design (Game Projects)

This section is verified against GitHub main @ 02ada853779588446254c6063637cbbca0a8bf37. The main and LTP trees contain the same 159 tracked WAV source files in KLTN/Assets/Audio; however, WAV file presence does not prove active AudioCatalog assignment or audible playback. The current main Unity audio runtime implements GameAudioRuntime, GameAudioEmitter, GameAudioSettings and environment-source routing. Sound assets and code pathways are described separately from listening, mixing, performance and multiplayer QA outcomes. Optional Purge audio is excluded from mandatory Expedition acceptance.

5.1 Audio Vision and Pillars

Audio vision: help the player infer danger and confirm accepted actions without exposing hidden Stalker state. The current main audio implementation separates local 2D UI/survival feedback from 3D positional world sources, and attaches emitters to observed interactions. Design priorities are legible threat cues, accurate gameplay outcomes, restrained industrial ambience and non-duplicated network presentation. Source inspection confirms code mappings; a hearing test on the accepted Windows build is still required.

Pillar	Description	Verification
Threat readability	Distinguish nearby Stalker motion/detection from distant ambient impacts	Two-player headphone listening test with occlusion/distance changes
Action feedback	Positive and negative objective, revive, shop and UI actions receive distinguishable cues	Gameplay/audio event checklist
Controlled repetition	Footsteps, warning one-shots and ambient events avoid obvious identical repetition	Listen during 15-minute representative session
Network consistency	Sound plays from accepted gameplay state without duplicated local+replicated one-shots	Host/client playback test
Resource efficiency	Clip count, import format and active voice budgets fit the Windows FPS target	Unity Profiler plus build-size/voice measurements

5.2 Platform and Audio Constraints

Item	main source observation / validation requirement
Platform/output	Unity Windows client using standard AudioSource playback; test headphone and stereo speaker output on the accepted build
Runtime components	GameAudioRuntime, GameAudioEmitter, GameAudioCatalog, GameAudioEnvironmentSource, GameAudioSettings and optional PurgeSpatialAudio
Volume controls	GameAudioSettings persists Master, Music and Effects via PlayerPrefs; Voice control is present but built-in voice chat is outside mandatory Review 2
Clip inventory	159 WAV files tracked under Assets/Audio in both main and LTP; not every clip is necessarily assigned/played by the runtime catalogue
3D attenuation	CreateSource(true) uses spatialBlend 1, minDistance 2 m, maxDistance 25 m and dopplerLevel 0; local/global sources use spatialBlend 0
Simultaneous voice count	NOT MEASURED: capture peak active AudioSource/voice count during multiplayer pursuit and final escape
RAM, loudness and installed footprint	NOT MEASURED: inspect Unity Audio Profiler, clip import settings and Windows build report

Runtime audio is presentation-only: authoritative game noise detection remains part of the AI/gameplay implementation, not an AudioSource waveform. GameAudioEmitter observes confirmed/networked state and suppresses the first snapshot, helping avoid retrospective sound playback. This does not certify absence of duplicate one-shots in all connection/rejoin cases; verify using host and client listening tests.

5.3 Audio Direction and References

The source implementation supports a quiet facility room tone, context-based relay/server/HVAC environment loops, footsteps, tension drones and short state-driven effects. The tone is sparse mechanical uncertainty rather than a continuous jump-scare soundtrack. Audible confirmation must follow accepted Host transitions for objectives, down/revive and match completion. The source Stalker presentation uses dedicated monster step/breath/growl and attack keys: do not replace them with arbitrary Purge creature clips.

We are	We are NOT
Low-frequency industrial tension and controlled space	Continuous loud horror stingers everywhere
Audible gameplay consequences	Sounds announcing success before Host/backend confirmation

We are	We are NOT
Repeat-aware, layered, spatial audio	A single obvious footstep or growl repeated unchanged
Readable danger cues	Music so loud that attacks, relays or teammates cannot be heard

5.4 Gameplay Sound Effects

Sound group	main GameAudioEmitter / asset evidence	Actual event mapping observed	Verification status
Footsteps	player/concrete_footstep_*, sprint_footstep_* and crouch_footstep_*	Movement state and speed used for rate/variant selection; default concrete selection does not prove automatic material recognition	Code mapped; inspect playback and local/remote attenuation
Flashlight	player/flashlight_on, player/flashlight_off	Flashlight IsOn state transition observed	Code mapped; avoid duplicate playback on host
Stalker	stalker/monster_growl_*, monster_scream_attack, miss_attack and monster_step_*	Replicated DETECT/CHASE/SEARCH/RECOVER, attack phase/outcome and movement drive 3D emitter	Code mapped; verify host/client and alert intelligibility
Energy core	energy_core/pickup, drop, deposit_into_sector_box	Network item state changes generate accepted pickup/drop/place cues	Code mapped; runtime verification required
Security terminal and relay	security_terminal/download_*, power_puzzle/correct_input, wrong_input	Terminal state/holders and puzzle progress/failure control one-shots and progress loop	Code mapped; interruptions/loop cleanup require test
First Aid / player life	first_aid/use_kit, revive_progress_loop, revive_complete, revive_cancel	Life-state transition ordinal, revive start/cancel/complete, local downed heartbeat	Code mapped; verify cancellation and persistence
Noise Maker	noise_maker/beacon_loop_loop	World beacon existence drives positioned loop	Code mapped; verify start/stop and replication
Final hunt/result	escape_endgame/final_hunt_start, countdown_tick, mission_success, mission_failure	NetworkMatchPhase and authoritative Win/Lose choose event cues	Code mapped; no premature win/duplicate result cues

The code-level playback paths above are source verification, not completed listening tests. In particular, the main Stalker emitter maps semantic states to monster_growl_* keys; the file stalker/detect_cue.wav being present does not prove that it is the active transition sound. Attacks must distinguish windup from authoritative hit/miss resolution. Avoid constant per-frame one-shots and check loop stop when an entity despawns, a revive is cancelled or a match ends.

5.5 Character, Environment and Ambience

Context	main source loop / cue	Code handling	Verification note
Facility base ambience	map_ambience/facility_room_tone_loop	GameAudioRuntime starts this in Lobby and the configured game scene	2D bed; check perceived level and startup
Relay A electrical area	map_ambience/electrical_room_loop	GameAudioEnvironmentSource auto-selects this room key on RelayAController	2D local room crossfade; verify active scene registration
Relay B server area	map_ambience/server_room_loop	GameAudioEnvironmentSource selects this key on RelayBController	Verify audible zone change, not a global per-material system
Security terminal/HVAC	map_ambience/hvac_loop	EnvironmentSource selects on SecurityTerminalDownload; nearest eligible source within 16 m	Room-source binding must be tested
Player survival	player/damage_grunt, downed_heartbeat_loop; first_aid/*	Emitter adjusts 2D local vs 3D remote state; heart/breath are local	Check heartbeat stops after recovery
Stalker and Purge extension	stalker/monster_breathing_* and footsteps; purge_* families separate	StalkerAudioController owns spatial chase audio where attached; optional Purge uses own runtime	Avoid duplicate 2D danger drone when StalkerController audio owns spatial chase

Main-specific detail: GameAudioRuntime checks active scene state at a one-second environment update interval and crossfades between the nearest eligible room sources. Room ambience is routed through local 2D AudioSources; world-object emitters are positional 3D. StalkerAudioController suppresses the additional 2D danger-drone path for Stalker prefabs already using the dedicated spatial chase loop. Do not claim full occlusion, reverb-zone simulation or automatic surface-aware footsteps without matching implementation and listening evidence.

5.6 Music

Main implements a sparse tonal danger layer rather than a fully scored soundtrack. GameAudioRuntime creates a separate dangerMusic AudioSource, which can play horror_ambience/tension_drone_loop when eligible Stalker danger is detected and the dedicated StalkerAudioController is absent; the logic deliberately avoids a second proximity drone when spatial Stalker chase audio is already responsible. The facility room tone is a separate bed. Imported warehouse demo music files do not establish active, cleared soundtrack tracks. Verify current MainMenu scene audio sources and licensing before listing any menu music as released content.

Design state	main source routing / candidate	Status
Main menu	Existing MainMenu.unity and GameAudioSettings route named BackgroundMusic AudioSources when present	Scene/source assignment and licence must be checked
Exploration	map_ambience/facility_room_tone_loop as base; local environment room keys	Mapped in GameAudioRuntime; listening test required
Stalker proximity	horror_ambience/tension_drone_loop under guarded proximity logic; spatial chase by StalkerAudioController	Conditional route in code, not guaranteed to play simultaneously
Final hunt	escape_endgame/final_hunt_start plus existing ambient/movement pressure	Event mapped in GameAudioEmitter; verify relative levels
Win/Lose	escape_endgame/mission_success and mission_failure	Mapped from authoritative match result; verify no premature emission

5.7 UI and Feedback Sounds

Event	Required audio semantics	Priority
Valid menu/room action	Short unobtrusive confirmation	Medium
Invalid login, join or protected action	Distinguishable denial, no success sound	High
Objective completion	Positive cue on accepted Host transition	High
Team damage/Downed/revive	Localized warning and medical progress/completion	Highest
Reward Pending	Neutral/pending feedback, not a success credit fanfare	Medium
Reward Confirmed / valid purchase	Positive cue after backend response	High
Rejected purchase or insufficient Credits	Denial cue with text and unchanged balance	High
End match Win/Lose	Distinct final outcome cues	High

GameAudioRuntime.UI and GameAudioButton support presentation-level UI sound playback on main. GameAudioEmitter maps lobby-ready, match-phase changes, objective progress and invalid interactions; API failure feedback uses the ui/error cue in specified paths. This does not prove all authentication, wallet, shop and legal-gate screens have complete auditory coverage. Until verified, treat payment/reward UI sound claims as designed semantics rather than executed acceptance evidence.

5.8 Voice-over and Localization

GameLanguage in main supports Vietnamese and English UI text. This is not proof that every audio cue or subtitle is translated. Mandatory Expedition does not require a recorded narrator or a VO dialogue collection. Voice settings and voice chat code exist but remain outside mandatory Review 2 acceptance; they must not be counted as completed localized voice-over production. For any future voiced lines, track transcript, locale, source and subtitle/accessibility coverage separately.

5.9 Mixing

Suggested priority: threat/damage and revive alerts first; accepted objective and final-result cues second; player actions third; environment SFX fourth; music/ambience last. The main source exposes separate Master/Music/Effects controls via GameAudioSettings and Music/Effects AudioMixer groups, with a special Voice setting for optional functionality. Proposed 6 dB ducking, LUFS targets or a maximum active-voice budget are not proven features unless confirmed in mixer settings and measured during listening/Profiler tests. World emitters should attenuate naturally; player-local feedback and UI should remain intelligible and not be doubled by network state listeners.

5.10 Technical Specifications

Audio source and runtime: Unity AudioSource and AudioMixer; source-generated WAV inventory; KLTN/Assets/Resources/GameAudioCatalog.asset; runtime processing in GameAudioRuntime, GameAudioEmitter, GameAudioSettings. docs/AUDIO_SETUP.md documents 2 m / 25 m world attenuation and no Doppler. Confirm the exact Unity import compression, sample rate, load mode and streaming settings in the Inspector before reporting them as final. The source KLTN/Assets/Audio/README.md describes synthesized mono PCM16, 44.1 kHz WAV clips for part of the catalogue; not all imported third-party audio must be assumed to share that format.

A WAV count is not equal to a bound/published audio catalog count. The audio_manifest.json list is partial relative to the complete Assets/Audio tree, and GameAudioCatalog.asset controls runtime lookup. Exact dBFS/LUFS peak, sample rate of every imported clip, RAM footprint and voice concurrency require tool-generated evidence rather than assumptions from file extensions.

Metric	main branch evidence / test requirement
Source asset count	159 tracked .wav files under KLTN/Assets/Audio (same contents as LTP at compared snapshots); active catalog inclusion must be inventoried separately
Audio RAM budget	NOT MEASURED — record current build Audio Profiler memory and clip loading
Installed audio footprint	NOT MEASURED — extract compressed asset/build report and compare with release target
Concurrent voices	NOT MEASURED — profile simultaneous AudioSource and mixer voice usage during 4-player hunt
Format/attenuation	Source README describes mono PCM16 44.1 kHz for synthesized clips; GameAudioRuntime 3D min 2 m, max 25 m, Doppler 0; inspect imports individually
Platform/codec acceptance	Windows PC only; validate headphone/speaker stereo and source import/compression in the final build

5.11 Audio Settings, Asset List and Schedule

Acceptance procedure: capture actual source SHA, Windows build/config, test date, participant count and playback evidence per high-priority sound event. Log missing or duplicated cues as defects; attach listening notes, video/audio clip references and Profiler screenshots. This chapter records main-branch implementation and planned verification, not blanket PASS results for F10 or production certification.

Category	Source location / implementation	Owner / acceptance action	Verified documentation status
Player and Stalker	Assets/Audio/player and stalker; GameAudioEmitter + StalkerAudioController	Unity audio/gameplay owner: record host/client playtest and clip mapping	159 source WAV total, event paths mapped; listening QA pending
Objective, medical and door	Assets/Audio/door, energy_core, first_aid, objectives, security_terminal	Unity gameplay owner: execute objective and revive cancellation cases	Source mapping in GameAudioEmitter; end-to-end QA pending
Rooms and industrial ambience	Assets/Audio/map_ambience, horror_ambience; EnvironmentSource + Runtime	Audio owner: check scene registration and 2D crossfade in current SciFi scene	Source room keys confirmed; mixing QA pending
UI, lobby and results	Assets/Audio/ui, escape_endgame; GameAudioButton/Emitter	UI owner: validate success/denial/pending and result cues	Some source mappings verified; all-screen coverage unverified
Music/tension	MainMenu scene audio source plus dangerMusic/tension_drone	Audio owner: inspect scene assignment, balance and third-party licence	Conditional danger layer in source; full soundtrack unverified
VO, language and voice	GameLanguage Vietnamese/English; optional voice code	UI owner: check text/VO/subtitle coverage separately	No required recorded VO confirmed; voice optional
Supplementary Purge	Assets/Audio/purge_*; PurgeSpatialAudio and PurgeVfx	Optional mode owner: separate QA only	Source extension only; no mandatory acceptance claim